

Neural Networks

Artificial Intelligence

Lecture Overview

- Motivation
- Biological and artificial neurons
- Single-layer perceptron
- Multi-layer perceptron
- Neural network design
- Neural network architectures

Lecture Overview

- **Motivation**
- Biological and artificial neurons
- Single-layer perceptron
- Multi-layer perceptron
- Neural network design
- Neural network architectures

Motivation

- Great ability of cognitive beings to carry out some tasks: shape recognition, speech and image processing, etc.
- It seemed important to understand and emulate successful mechanisms from humans and animals: Parallelism and high connectivity.
- A branch of artificial intelligence: **Artificial Neural Networks.**

Motivation

- New paradigm (non-algorithmic) to process information (neurocomputing): learning and adaptation, distributed and parallel processing.
- New computational tools (faster and cheaper)
- In the future: more caution and theoretical support. Open issue: Generalisation.

Motivation

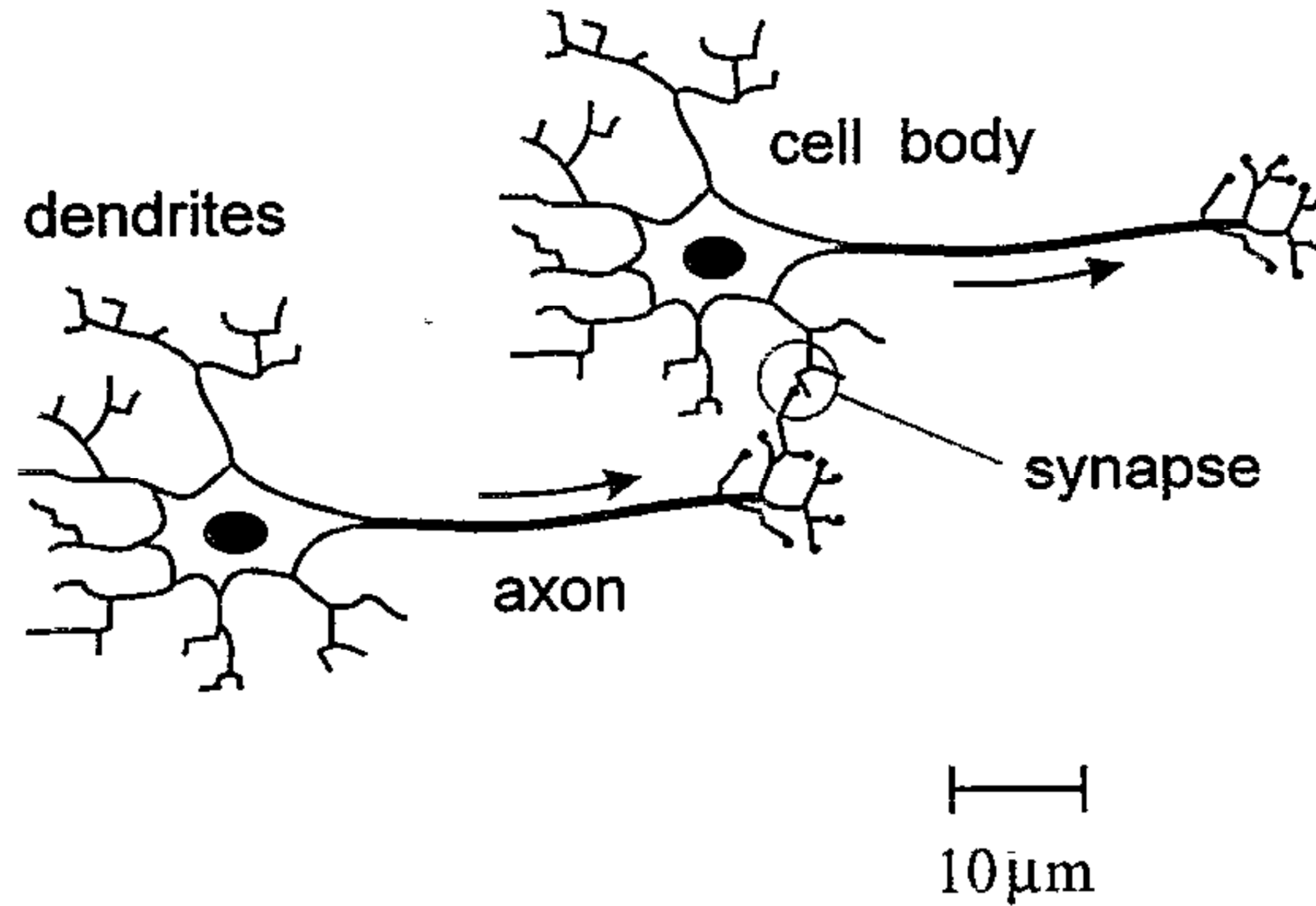
- The general problem of function approximation can be divided into two subproblems:
 - **Classification:** to approximate a function that represents the membership function of an entity – characterized by a set of input variables, either continuous or discrete – to a particular class (output with discrete values), e.g., character recognition.
 - **Regression:** to approximate the generating function (unknown) of a process by mapping elements from the input variables to output variables. Usually, continuous values are used.

$$y = f(x, w)$$

Lecture Overview

- Motivation
- **Biological and artificial neurons**
- Single-layer perceptron
- Multi-layer perceptron
- Neural network design
- Neural network architectures

Biological Neuron



Biological Neuron

- The brain is made up of **neurons** (nerve cells) which have
 - a cell body (soma)
 - **dendrites** (inputs)
 - an **axon** (outputs)
 - **synapses** (connections between cells)
- Synapses can be **excitatory** or **inhibitory** and may change over time.
- When the inputs reach some threshold an **action potential** (electrical pulse) is sent along the axon to the outputs.

Biological Neuron

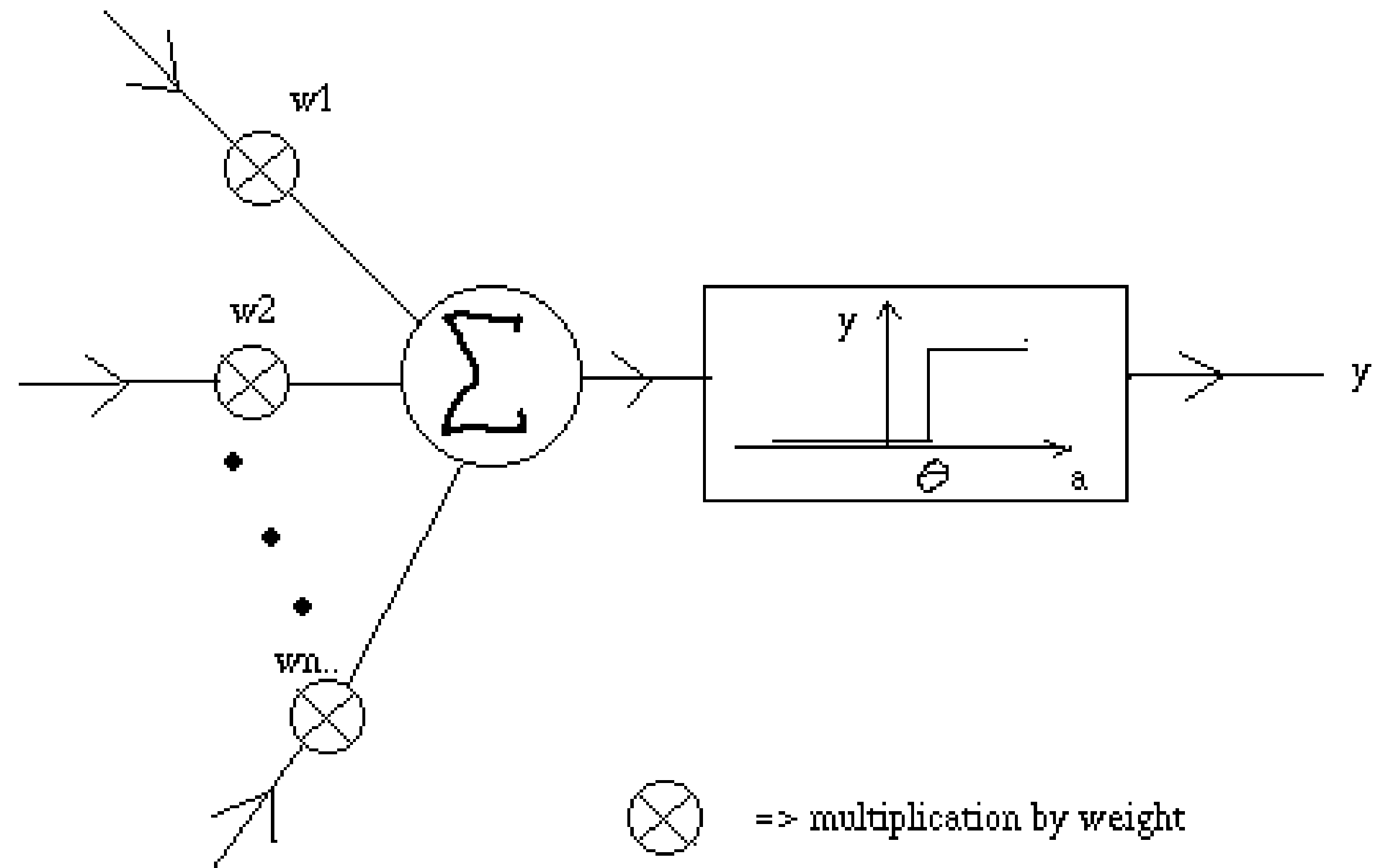
- Human brain has 100 billion neurons ($\sim 10^{10}$ - 10^{11} neurons) with an average of 10,000 synapses each (some even with 100,000 synapses).
- Latency is about 3-6 milliseconds.
- At most a few hundred “steps” in any mental computation, but **massively parallel**.

Artificial Neuron

- Automata characterised by:
 - An internal state.
 - Input signals.
 - Activation and transfer functions.

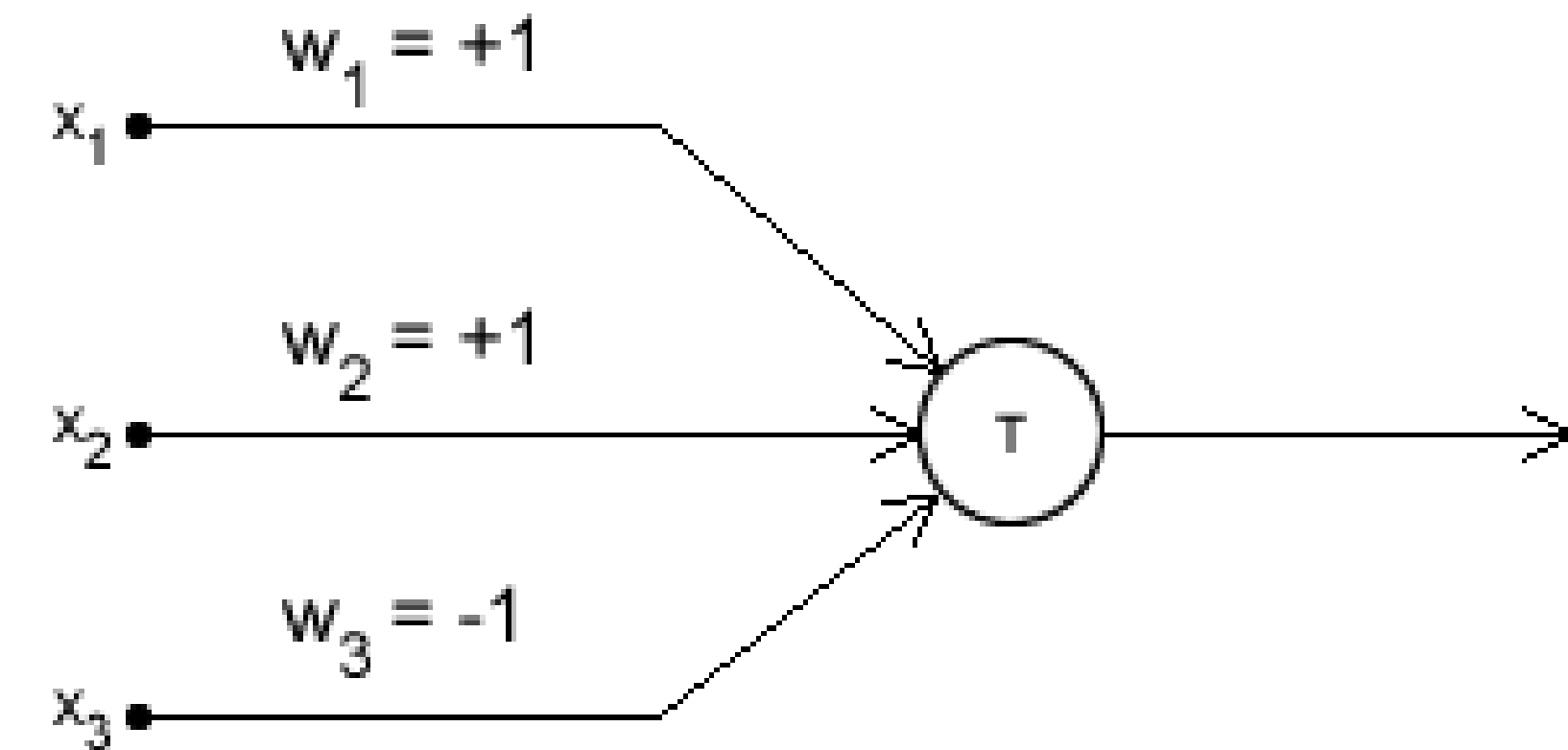
Artificial Neuron

- McCulloch-Pitts' model (1943)



Artificial Neuron

- McCulloch-Pitts model:
 - Inputs either 0 or 1.
 - Output 0 or 1.
 - Input can be either excitatory or inhibitory.
- Summing inputs
 - If input is 1, and is excitatory, add 1 to sum.
 - If input is 1, and is inhibitory, subtract 1 from sum.
- Threshold,
 - if $\text{sum} < \text{threshold } \Theta$, output 0.
 - Otherwise, output 1.



$$\text{sum} = x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3 + \dots$$

if $\text{sum} < \Theta$ **then** *output* is 0
else *output* is 1.

Learning

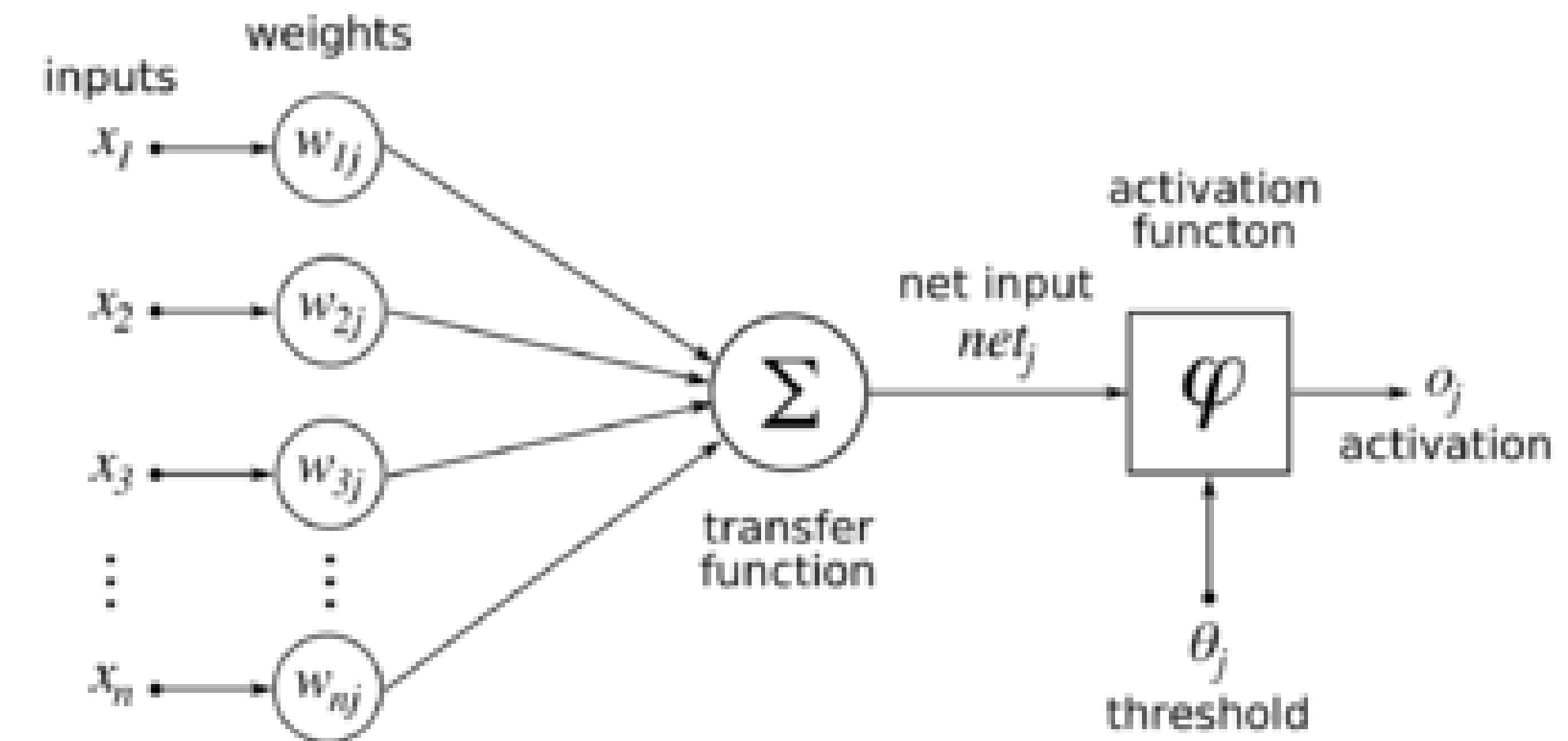
- Ability of a neuron (or neural net) to adjust connections (weights) to obtain the intended output or that meets certain criteria.
- **Hebbian learning (1949):** When a neuron A persistently activates another nearby neuron B, the connection between the two neurons becomes stronger. Specifically, a growth process occurs that increases how effective neuron A is in activating neuron B. As a result, the connection between those two neurons is strengthened over time.
 - “Neurons that fire together, wire together”, Hebb.

Artificial Neural Networks

- Information processing architecture loosely modelling the brain
- Consists of many interconnected **processing units (neurons)**
 - Work in parallel to accomplish a global task
- Generally used to model relationships between inputs and outputs or to find patterns in data
- Characterized by (i) number of neurons, (ii) interconnection architecture, (iii) weight values, (iv) activation and transfer functions.

Artificial Neural Networks

- ANNs nodes have
 - inputs edges with some **weights**
 - outputs edges with **weights**
 - **activation level** (function of inputs)



- Weights can be positive or negative and may change over time (learning).
- The **input function** is the weighted sum of the activation levels of inputs.
- The activation level is a non-linear **transfer** function g of this input:

$$\text{activation}_j = g(s_j) = g\left(\sum_i w_{ij}x_i\right)$$

Some nodes are inputs (sensing), some are outputs (action)

Artificial Neural Networks

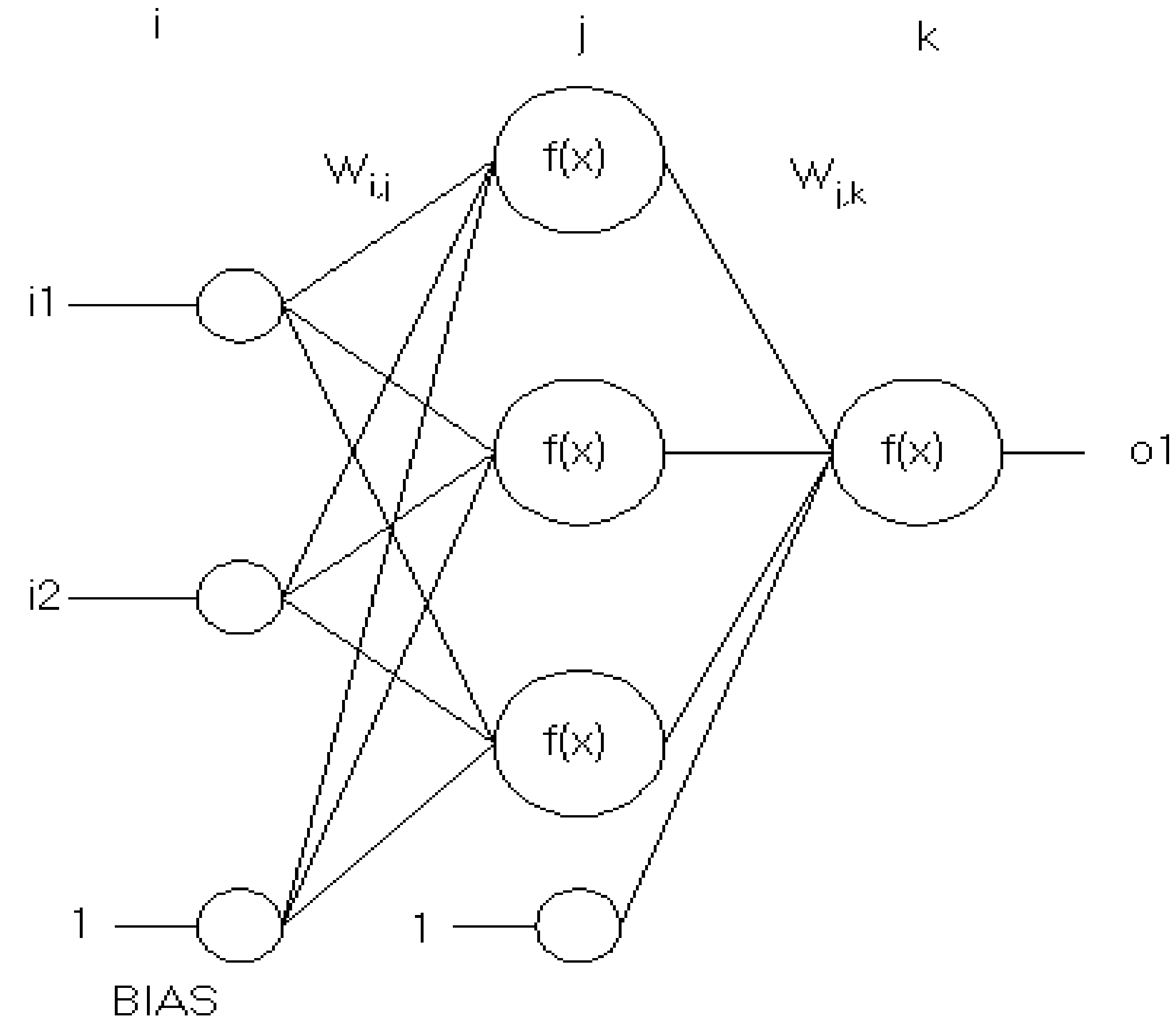


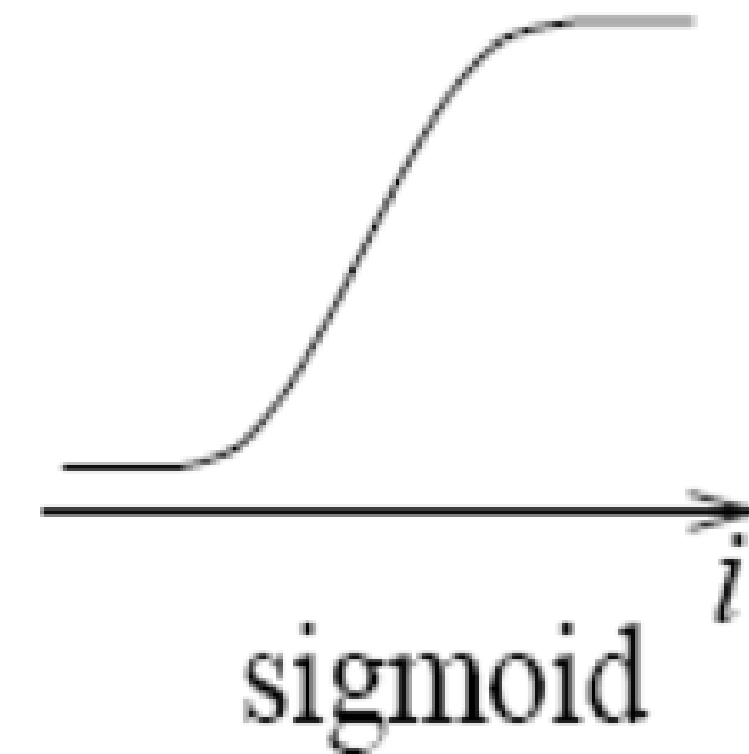
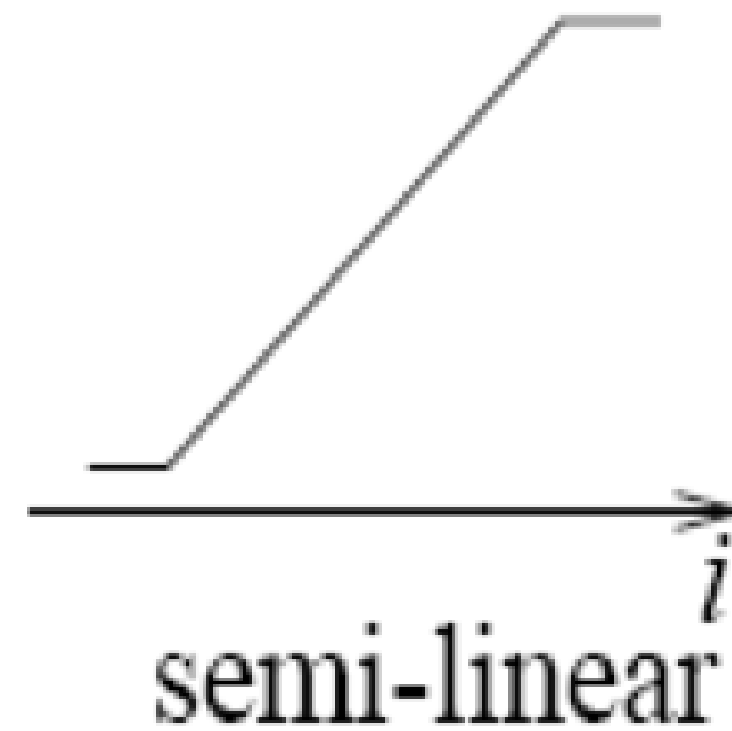
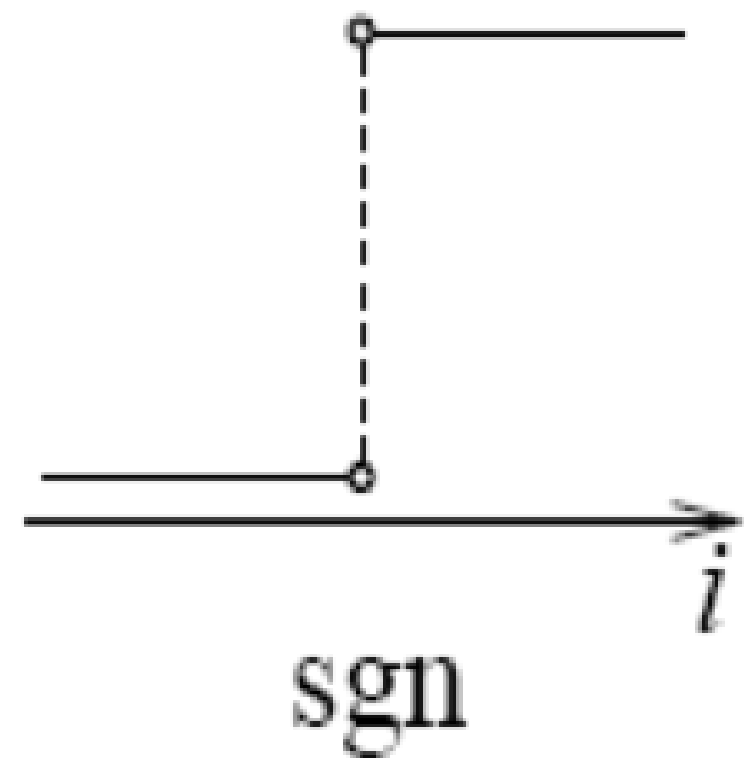
Figure - 1 A simple 2x3x1 NN. The lines connecting the neurons represent the weights. Also shown are the bias nodes used to shift the neuron transfer function and improves the network performance.

Artificial Neural Networks

- Neural networks (NN) might work in two ways:
 - **Learning:** adapting its weights, architecture, activation and transfer functions.
 - **Simulation or recognition:** it is used for information processing.
- NN learning: Supervised (through examples), unsupervised.

Activation Functions

Function $g(s)$ takes the weighted sum of inputs and produces output for node, given some threshold.



$$g(s) = \begin{cases} 1 & \text{if } s \geq 0 \\ 0 & \text{if } s < 0 \end{cases}$$

Lecture Overview

- Motivation
- Biological and artificial neurons
- **Single-layer perceptron**
- Multi-layer perceptron
- Neural network design
- Neural network architectures

Single-layer Perceptron

- Frank Rosenblatt, 1957
 - Use a logic threshold function for classification tasks.
- Classification and discriminant functions
 - How can we make the classification?
 - Using discriminant functions defined in the vector space we want to classify and produce a value that can be compared.

Single-layer Perceptron

- If we have a function (g_i) for each class (s_i). The classification rule is:

$$u \in s_i \text{ iff } g_i(u) > g_j(u); j \neq i$$

- For 2-classes problems, it can be reduced to one function as:

$$g(u) = g_1(u) - g_2(u), \text{ then } u \in s_1 \text{ if } g(u) > 0 \text{ and } u \in s_2 \text{ otherwise}$$

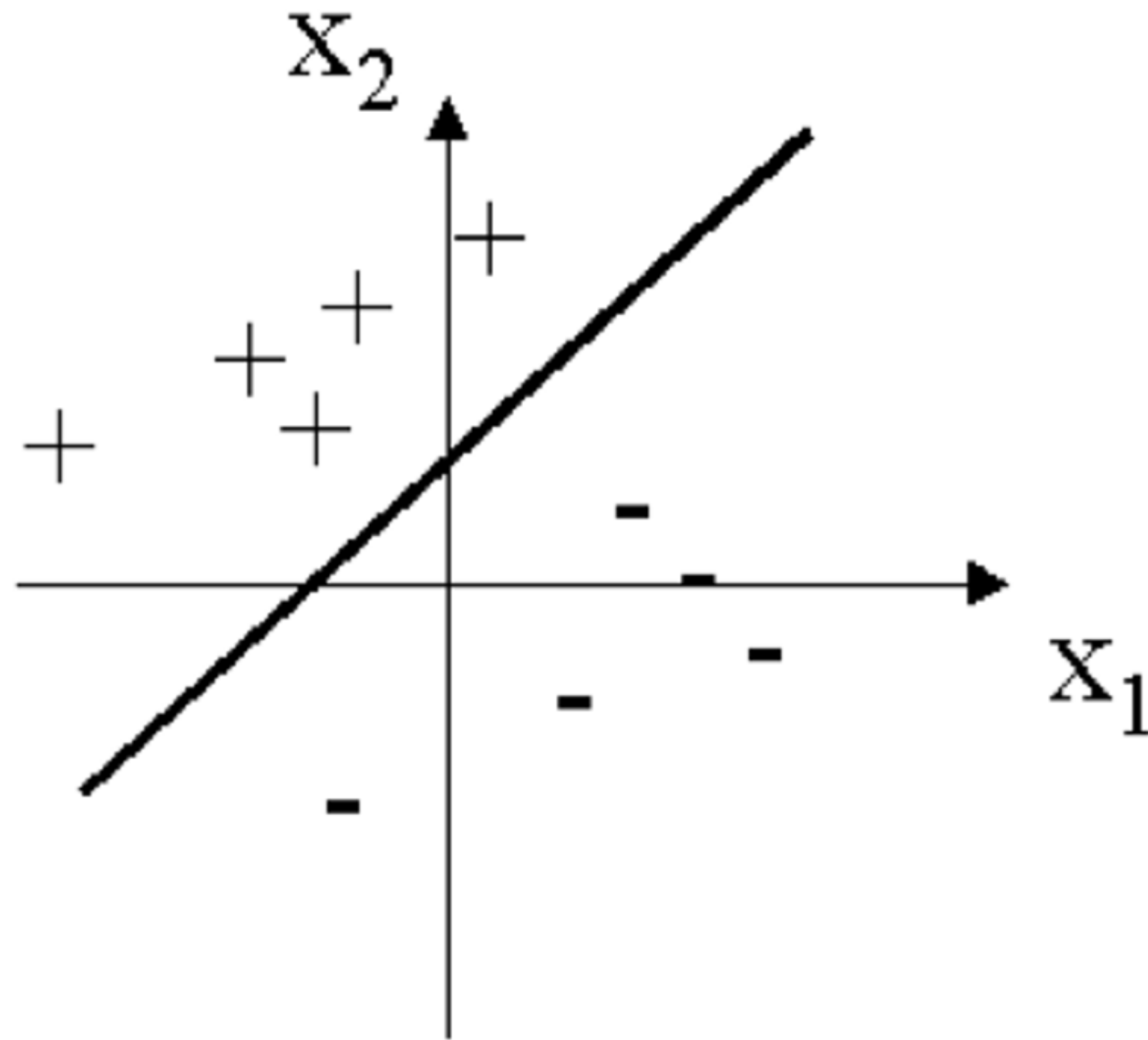
Single-layer Perceptron

- Linear classification with discriminant function using perceptron:

$$g(u) = w_1 u_1 + w_2 u_2 + \dots + w_n u_n = w \cdot u$$

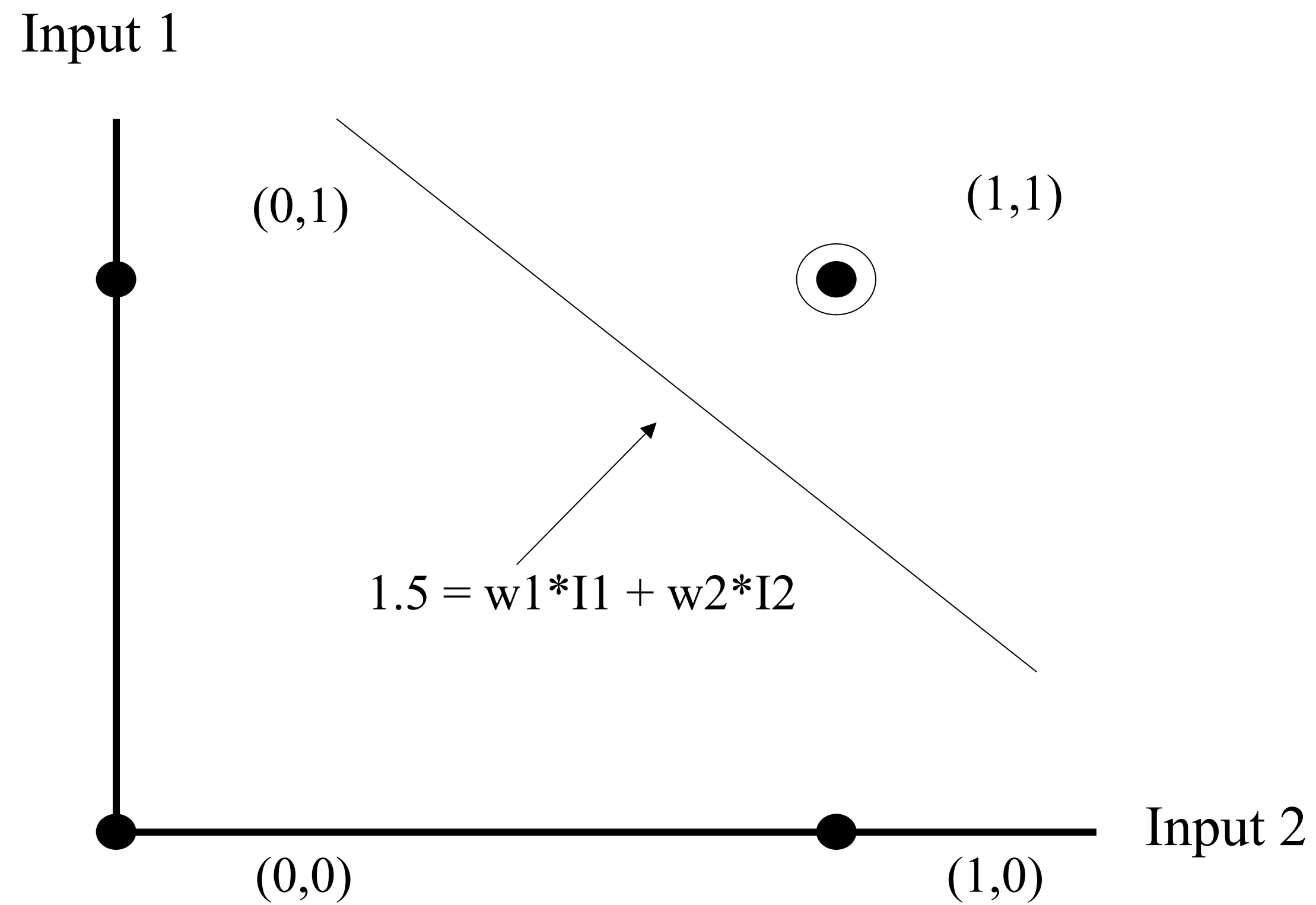
- The value associated with $g(u) = 0$ corresponds to the border, a hyperplane, that divides the two classes.
- Perceptrons are appropriate only for classes linearly separable.
- The learning problem is reduced to finding a hyperplane that separates the two classes.

Single-layer Perceptron



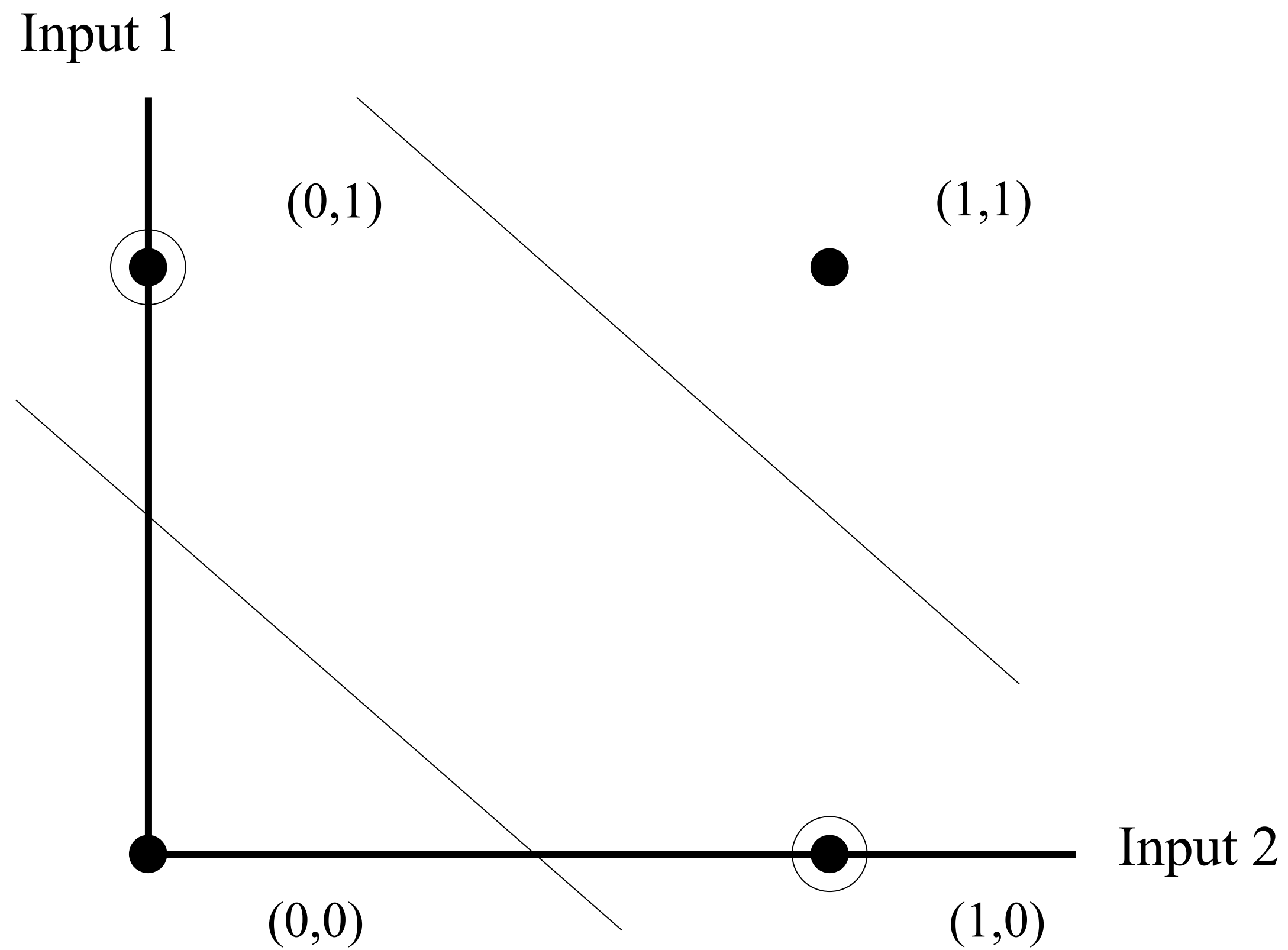
Single-layer Perceptron

- AND gate output



Single-layer Perceptron

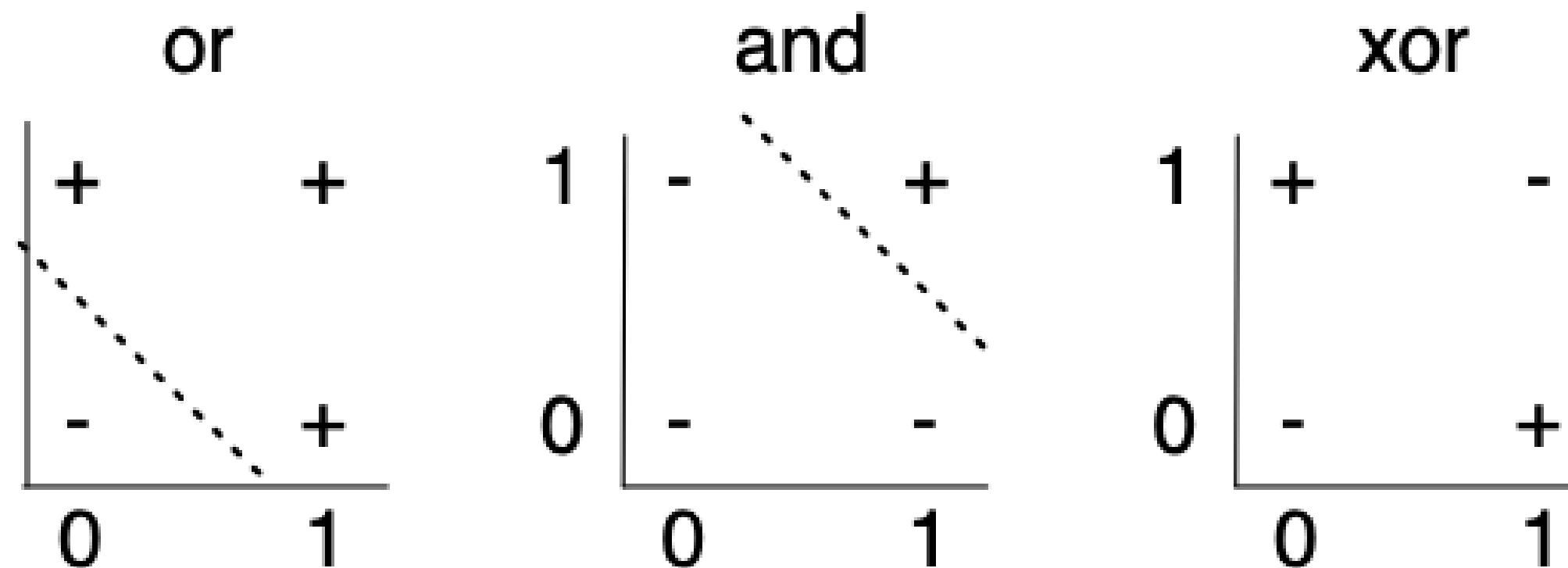
- XOR gate
- $\Rightarrow$ hidden layer needed



Single-layer Perceptron

- **Linearly separable** if there is a hyperplane where classification is true on one side of the hyperplane and false on the other side
- For the sigmoid function, when the hyperplane is:

$$x_1 \cdot w_1 + \cdots + x_n \cdot w_n = 0$$



Single-layer Perceptron

- Learning rule

$\hat{y}(x) = g(X \cdot W^t) \rightarrow$ perceptron output
 $y(x) \rightarrow$ target output $\in \{-1, 1\}$

$$\Delta W_k = \alpha(y(x_k) - \hat{y}(x_k)) \cdot x_k$$

$$W_{k+1} = W_k + \Delta W_k$$

$\Delta W_k = 0 \rightarrow$ if the perceptron output is correct

$\Delta W_k = +2\alpha x_k \rightarrow$ if $y(x_k) = 1$ and $\hat{y}(x_k) = -1$

$\Delta W_k = -2\alpha x_k \rightarrow$ if $y(x_k) = -1$ and $\hat{y}(x_k) = 1$

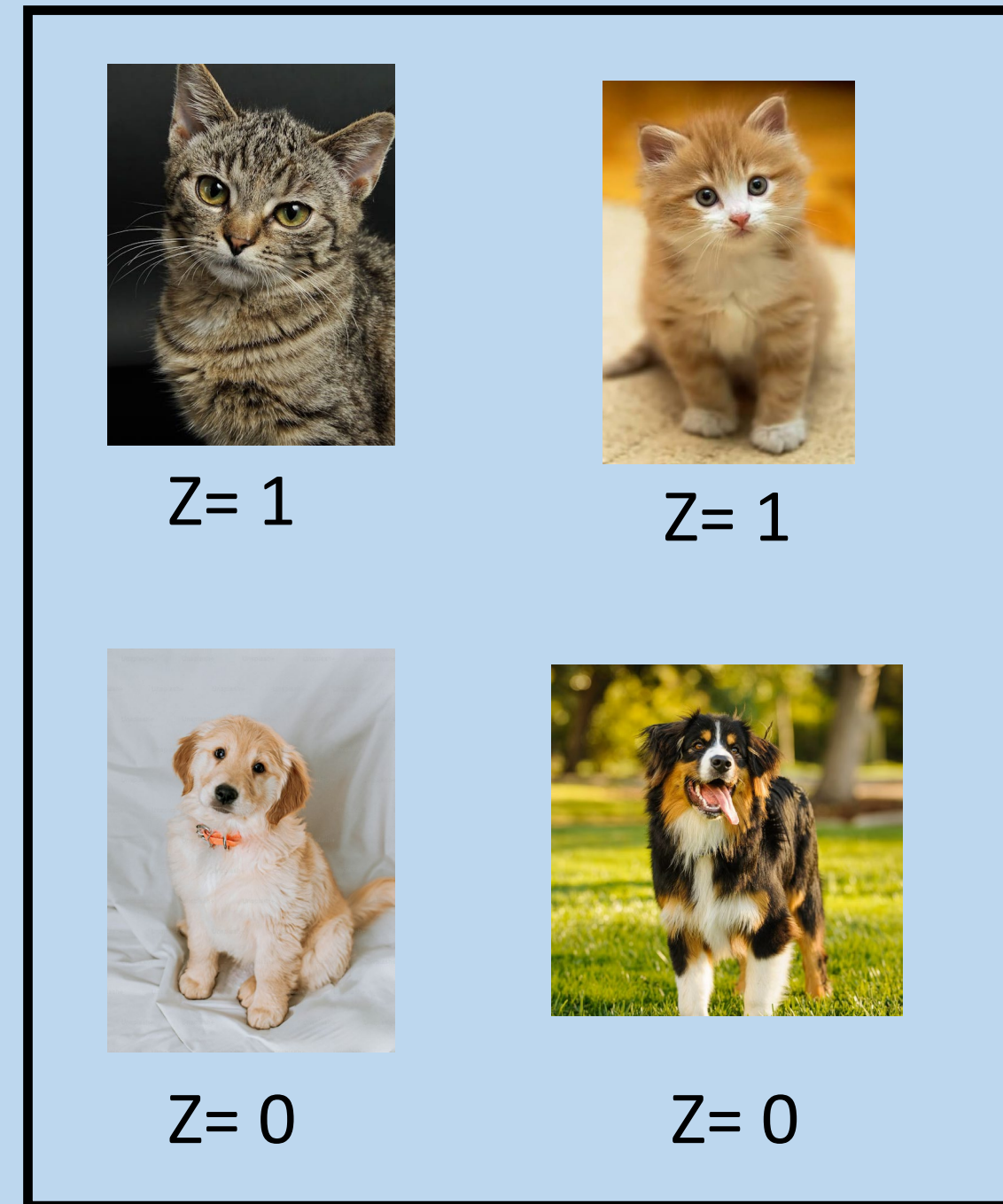
$\alpha > 0$; $\{x_1, \dots, x_k\}$ input sequence

Single-layer Perceptron

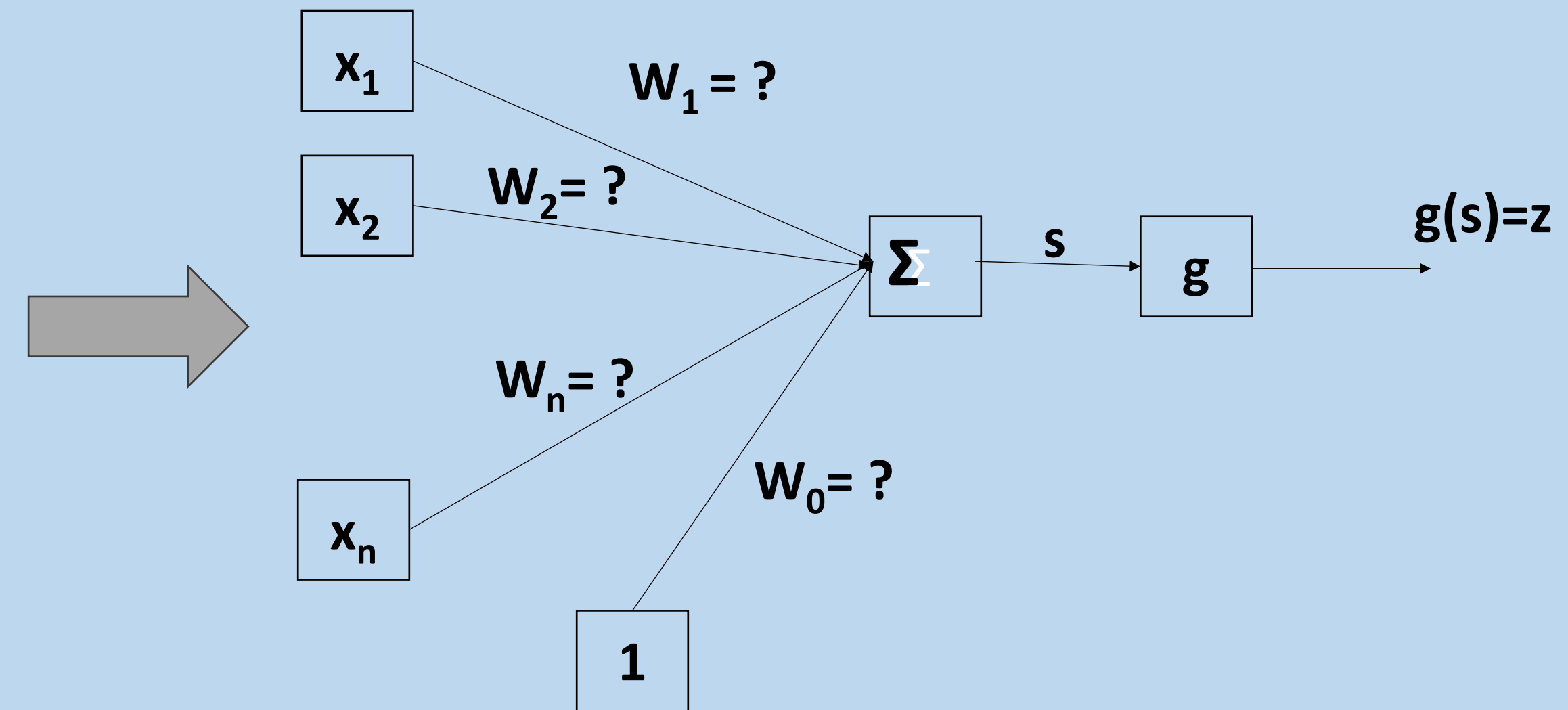
- Perceptron convergence theorem:
 - For any data set that is linearly separable, the perceptron learning rule is guaranteed to find a solution in a finite number of iterations.

Neural Networks, Perceptron Learning

In learning, the network finds the right w_i values in order to make the right prediction.



Dataset

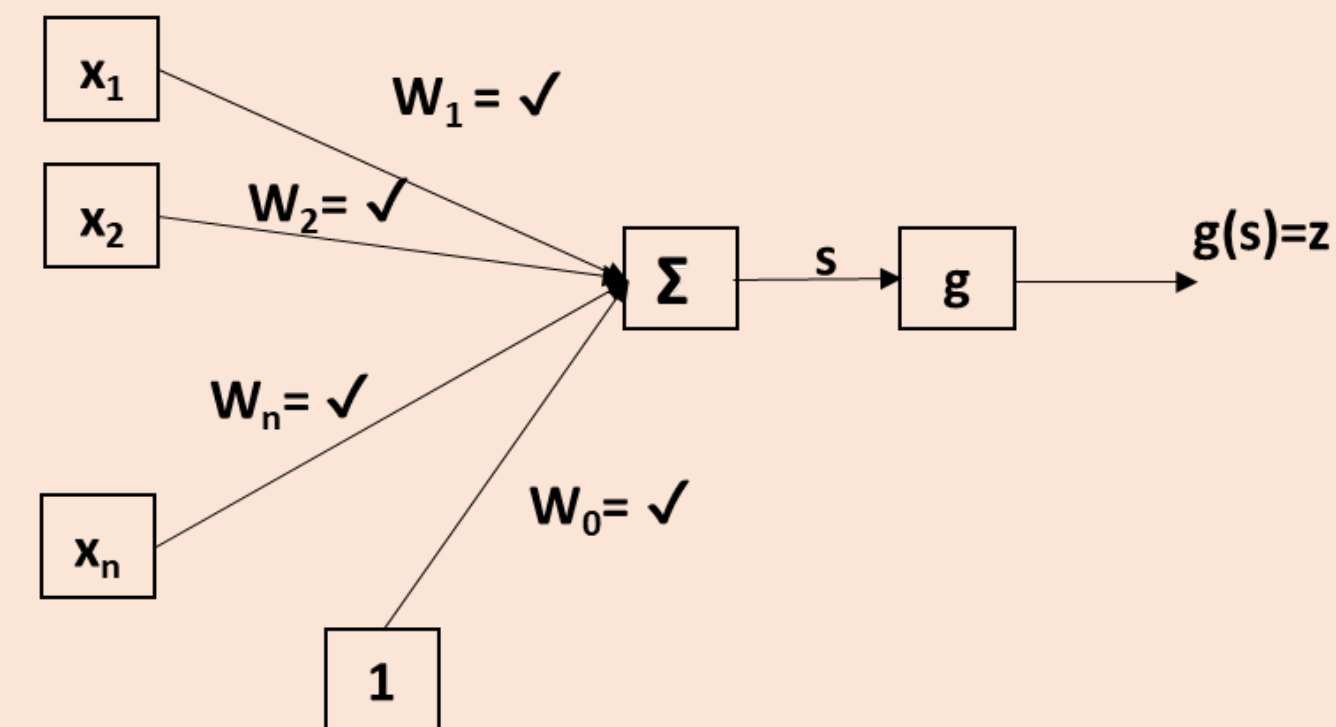
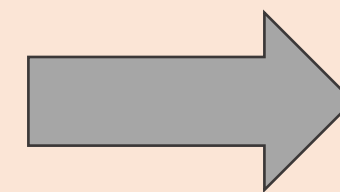


Learning Phase

After training, based on w_i values, model can make a prediction for a new unseen data.



$Z = ?$



$Z = 1$

After Learning

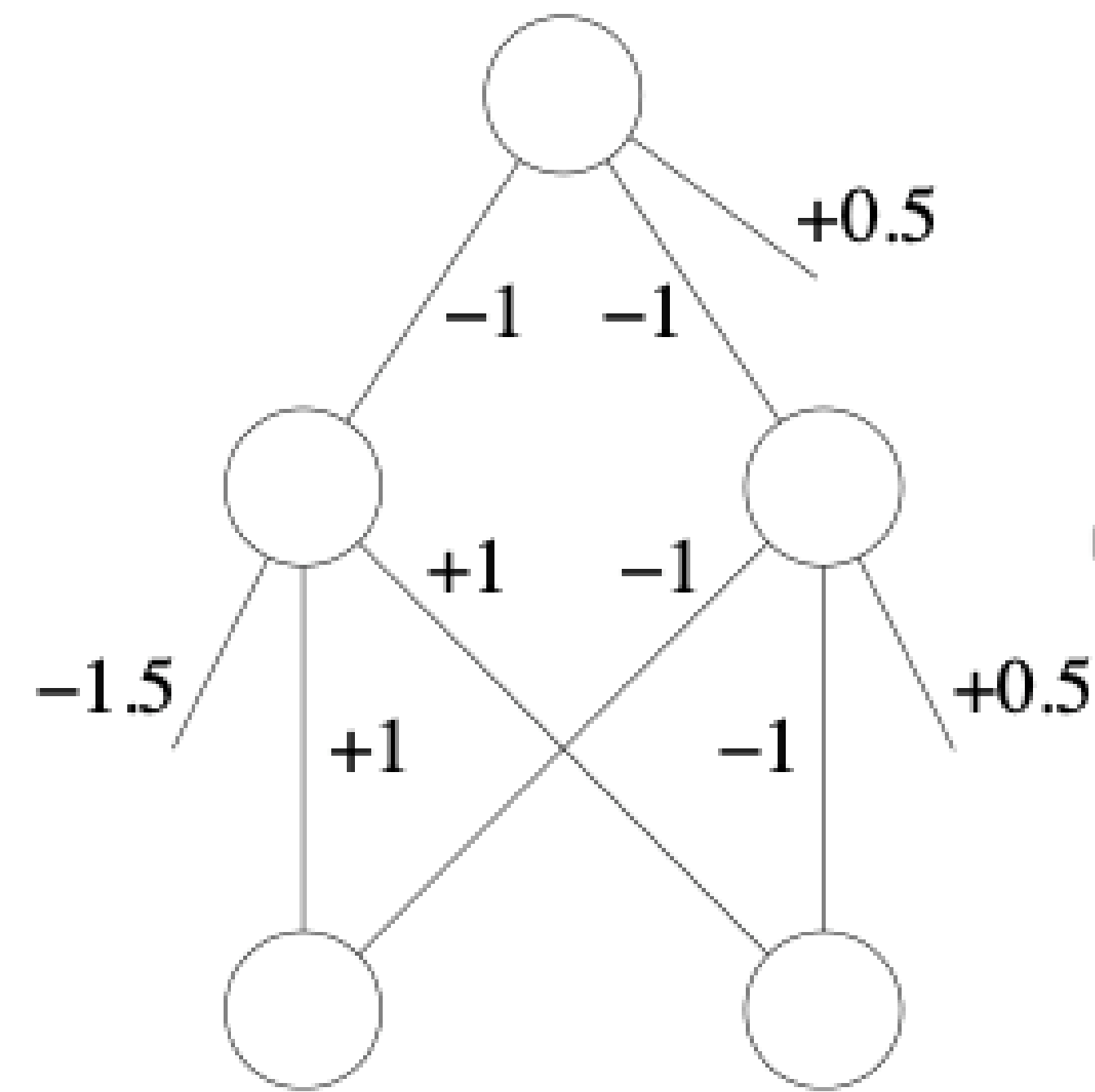
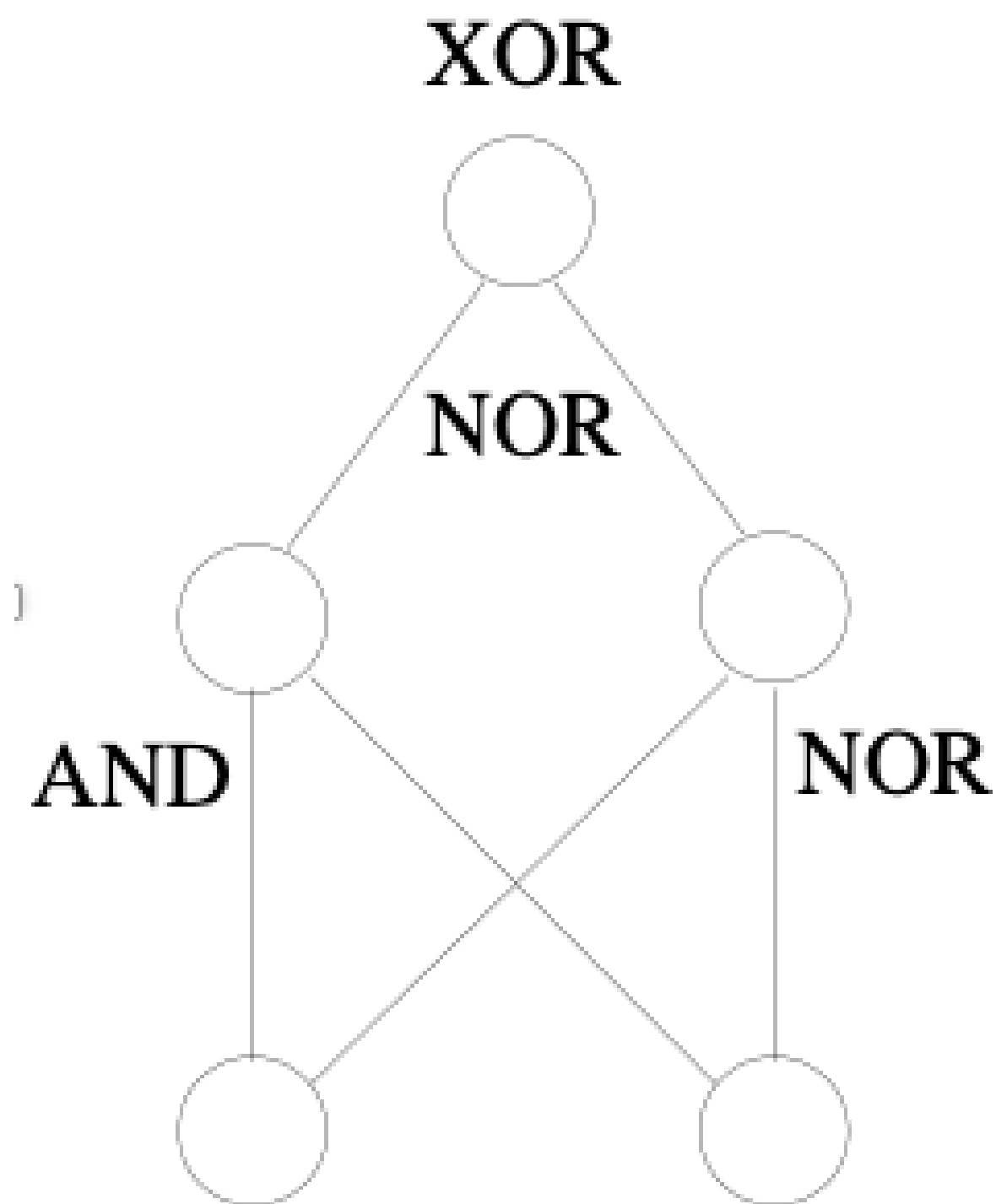
Historical Context

- In 1969, Minsky and Papert published a book highlighting limitations of perceptrons.
 - Funding agencies redirected funding away from neural network research preferring instead logic-based methods such as expert systems.
- Known since 1960s that any logical function could be implemented in a 2-layer neural network with step function activations.
- The problem was how to learn the weights of a multi-layer neural network from training examples.
- Solution found in 1974 by Paul Werbos.
 - Not widely known until rediscovered in 1986 by Rumelhart, Hinton and Williams.

Lecture Overview

- Motivation
- Biological and artificial neurons
- Single-layer perceptron
- **Multi-layer perceptron**
- Neural network design
- Neural network architectures

Multi-layer Neural Network



- Given an explicit logical function, we can design a multi-layer neural network by hand to compute that function.
- But, if we are just given a set of training data, can we train a multi-layer network to fit these data?

Multi-layer Perceptron

- **Definition:** A network in which its neurons are organised in successive layers. Each layer receives inputs from the previous one (or external input) and sends its outputs to the next layer. There are no internal connections in each layer.

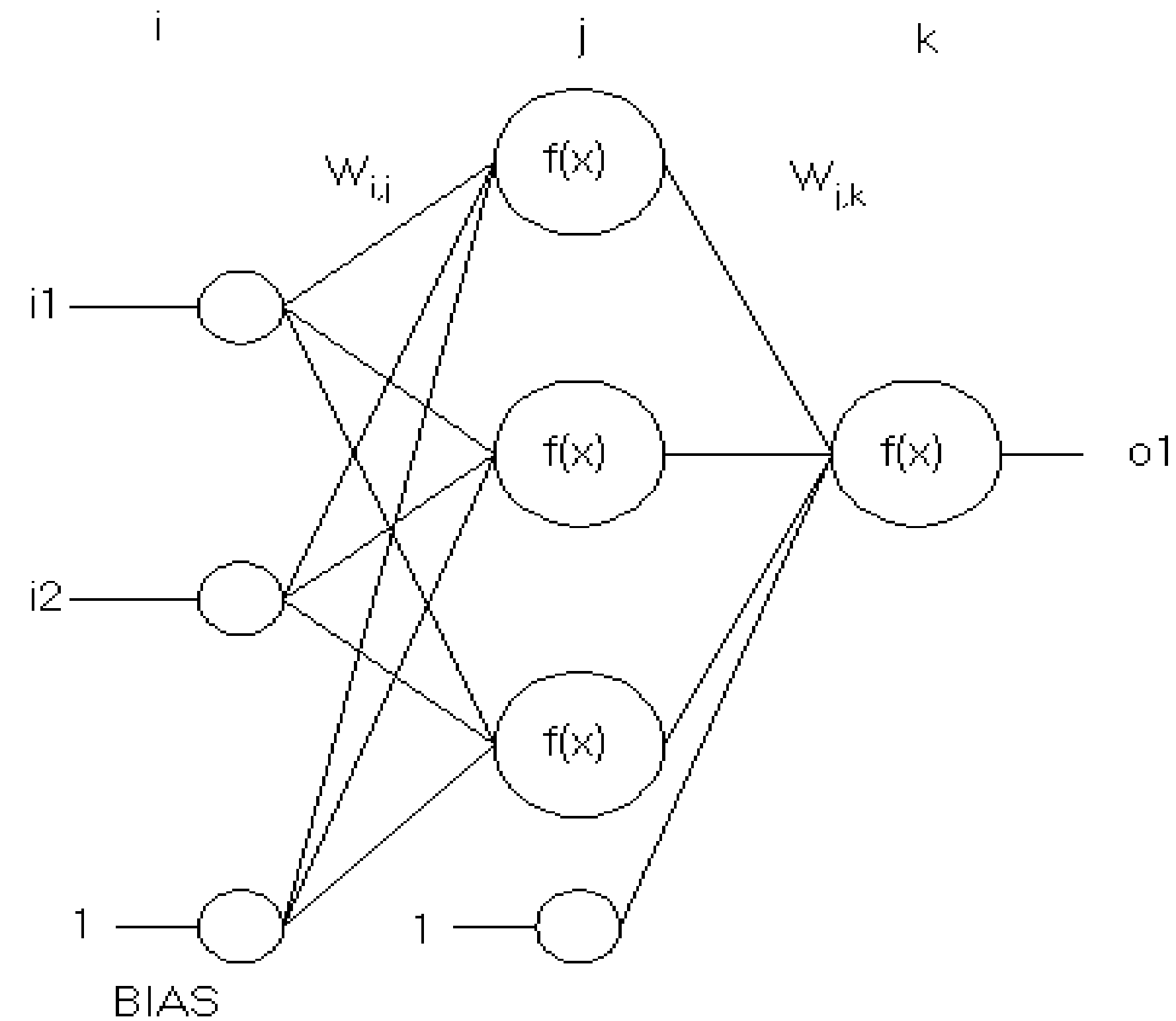
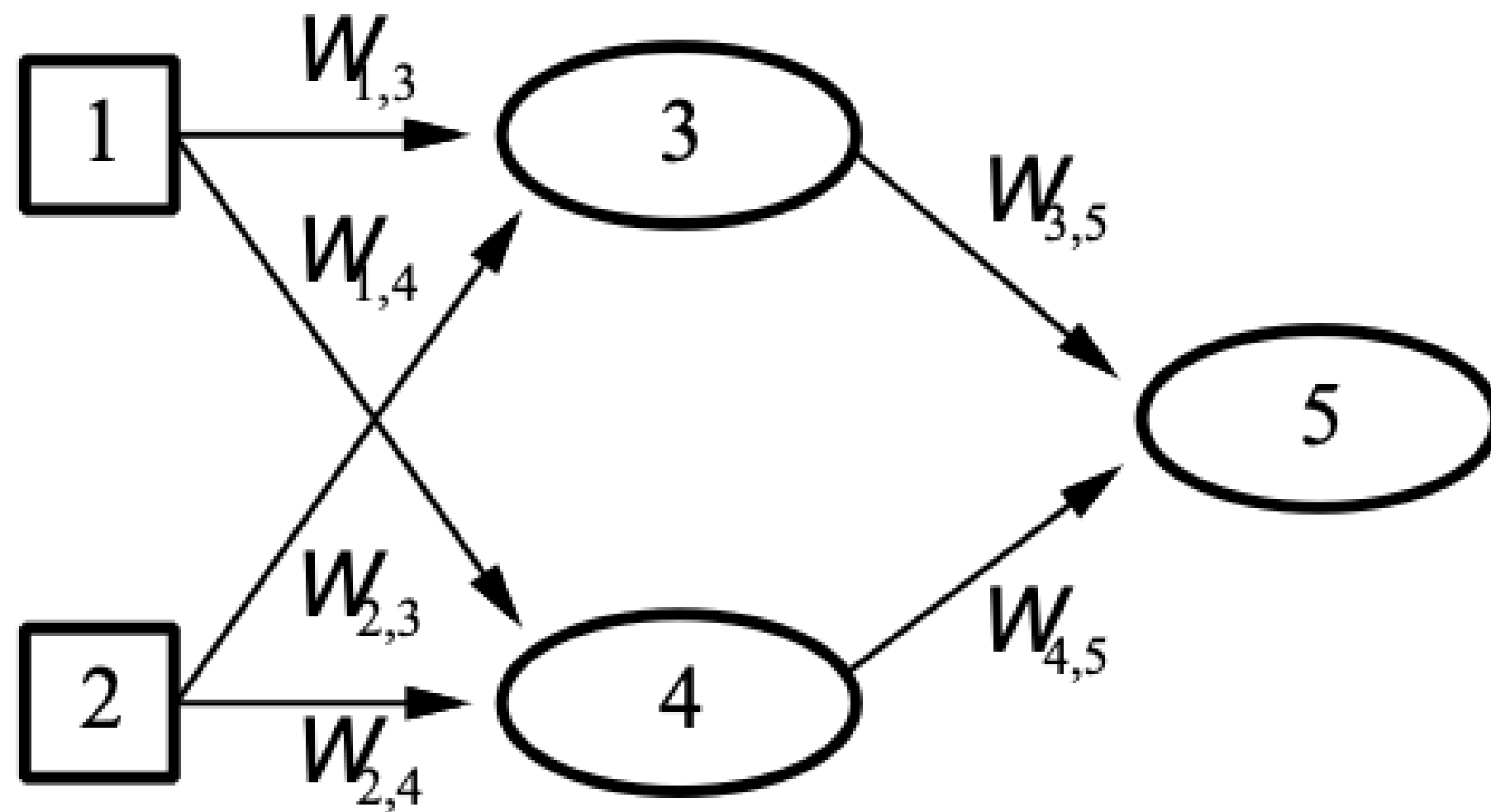


Figure - 1 A simple 2x3x1 NN. The lines connecting the neurons represent the weights. Also shown are the bias nodes used to shift the neuron transfer function and improves the network performance.

Feedforward Propagation



$w_{i,j} \equiv$ weight between node i and node j

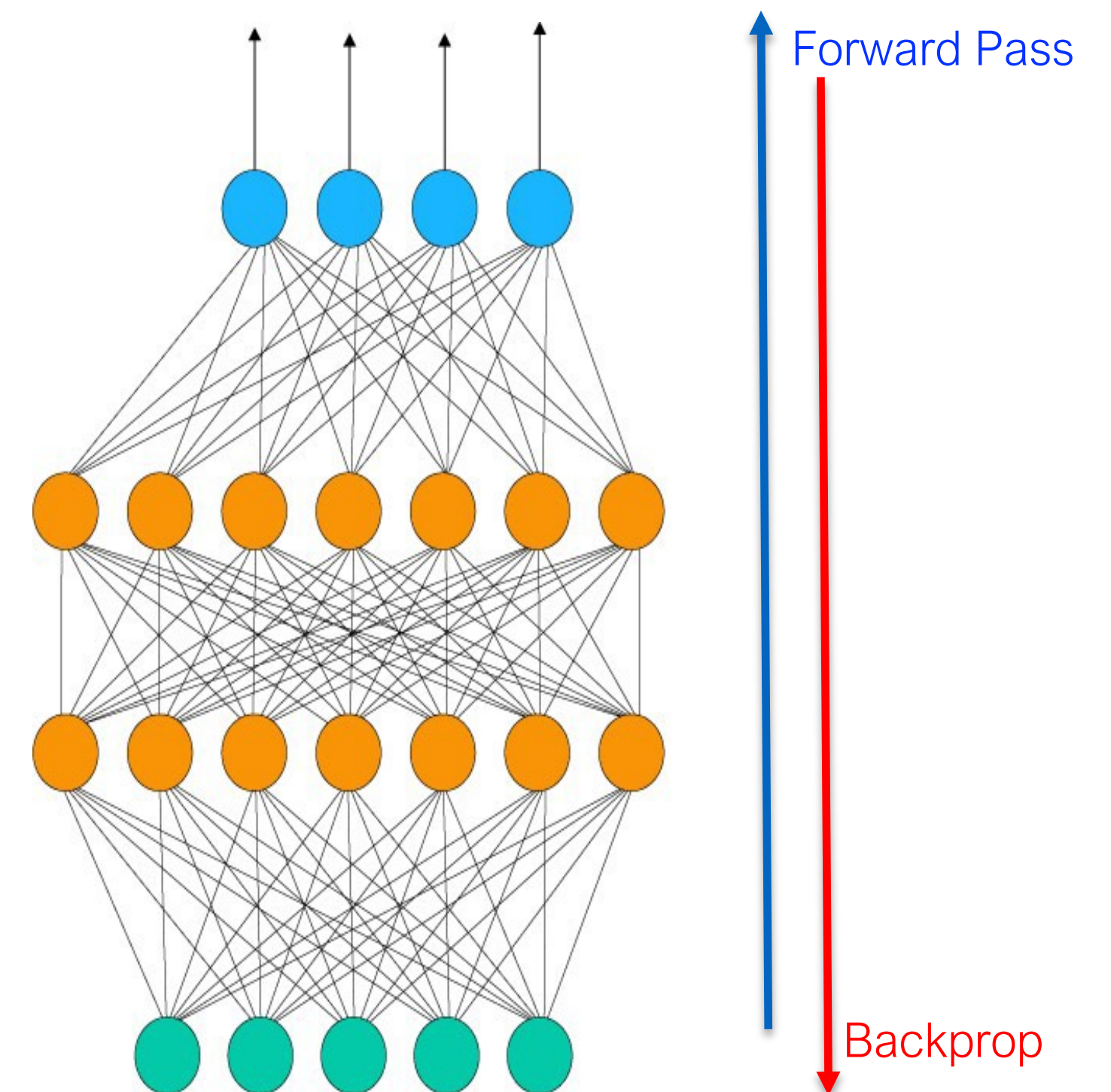
- Feed-forward network = a parameterised family of nonlinear functions:

$$\begin{aligned} a_5 &= g(W_{3,5} \cdot a_3 + W_{4,5} \cdot a_4) \\ &= g\left(W_{3,5} \cdot g(W_{1,3} \cdot a_1 + W_{2,3} \cdot a_2) + W_{4,5} \cdot g(W_{1,4} \cdot a_1 + W_{2,4} \cdot a_2)\right) \end{aligned}$$

- Adjusting weights changes the function

Backpropagation

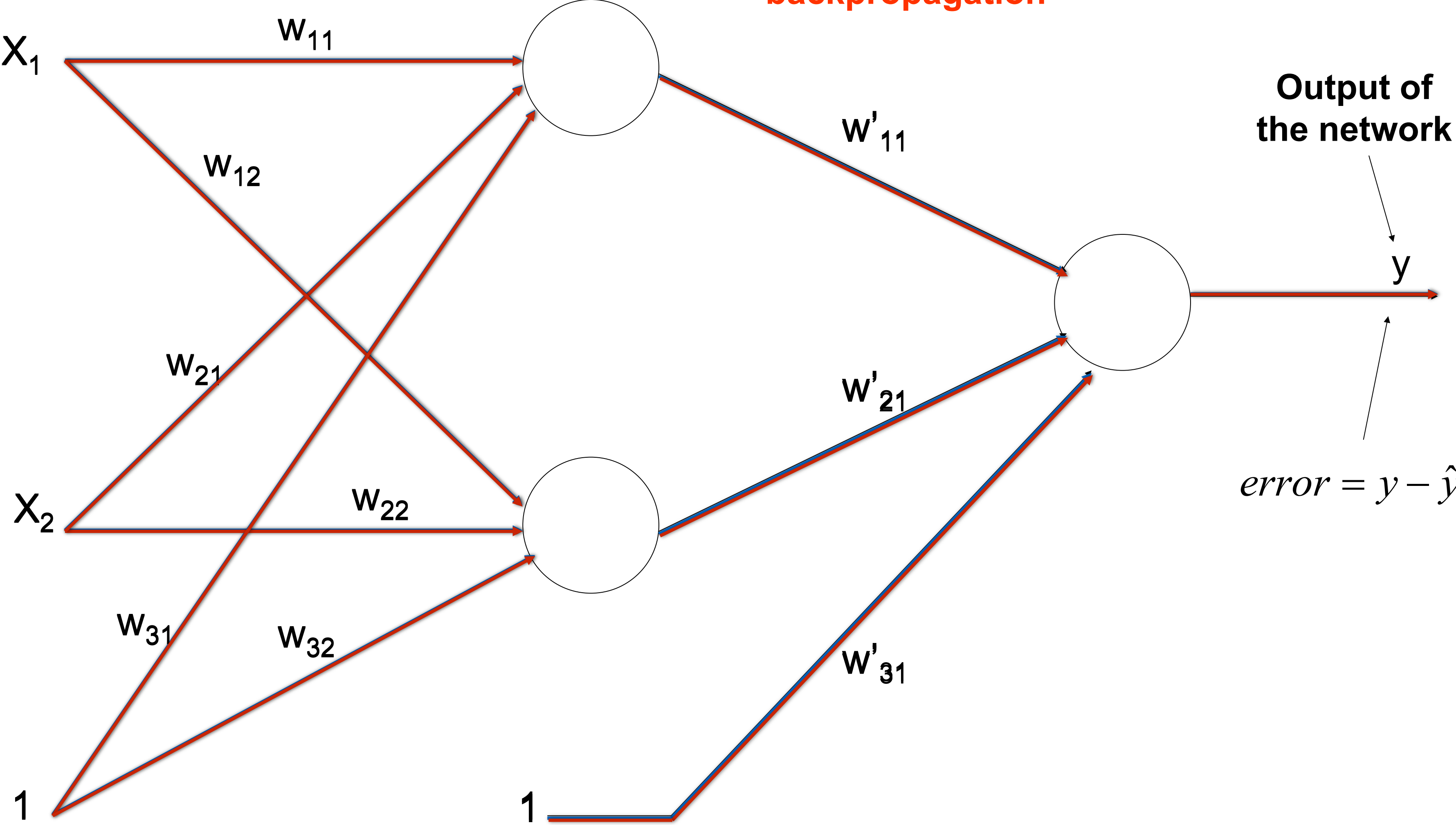
1. **Forward pass:** apply inputs to the “lowest layer” and feed activations forward to get output.
2. **Calculate error:** difference between desired output and actual output.
3. **Backward pass:** Propagate errors back through the network to adjust weights.



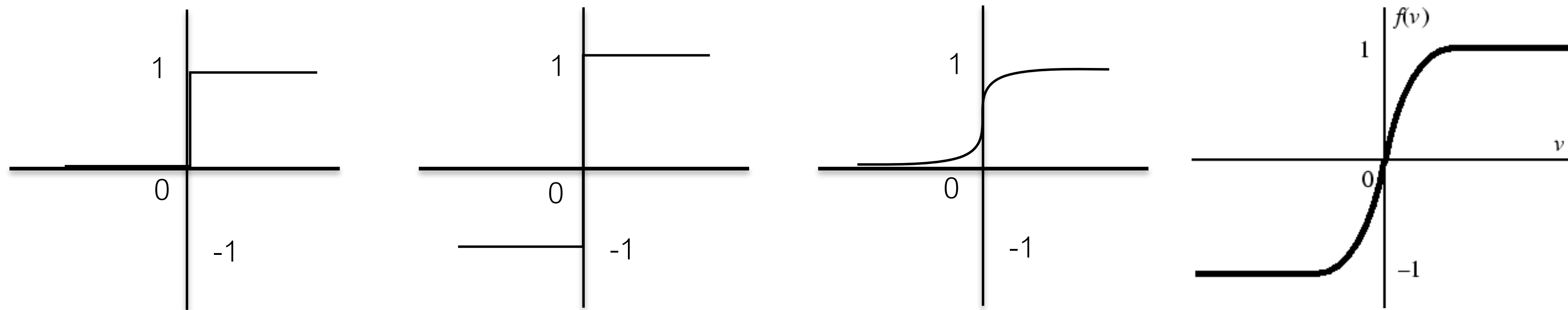
Backpropagation

Updating weights
of the network

Error
backpropagation



Backpropagation



Function must be continuous to be differentiable

Replace the (discontinuous) step function with a differentiable function, such as the sigmoid:

$$g(s) = \frac{1}{1 + e^{-s}}$$

or hyperbolic tangent

$$g(s) = \tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}} = 2 \left(\frac{1}{1 + e^{-2s}} \right) - 1 \quad (-1 \text{ to } 1)$$

Backpropagation

Gradient descent

$$E = \frac{1}{2N} \sum (d - y)^2$$

If transfer functions are smooth, can use multivariate calculus to adjust weights by taking the steepest downhill direction.

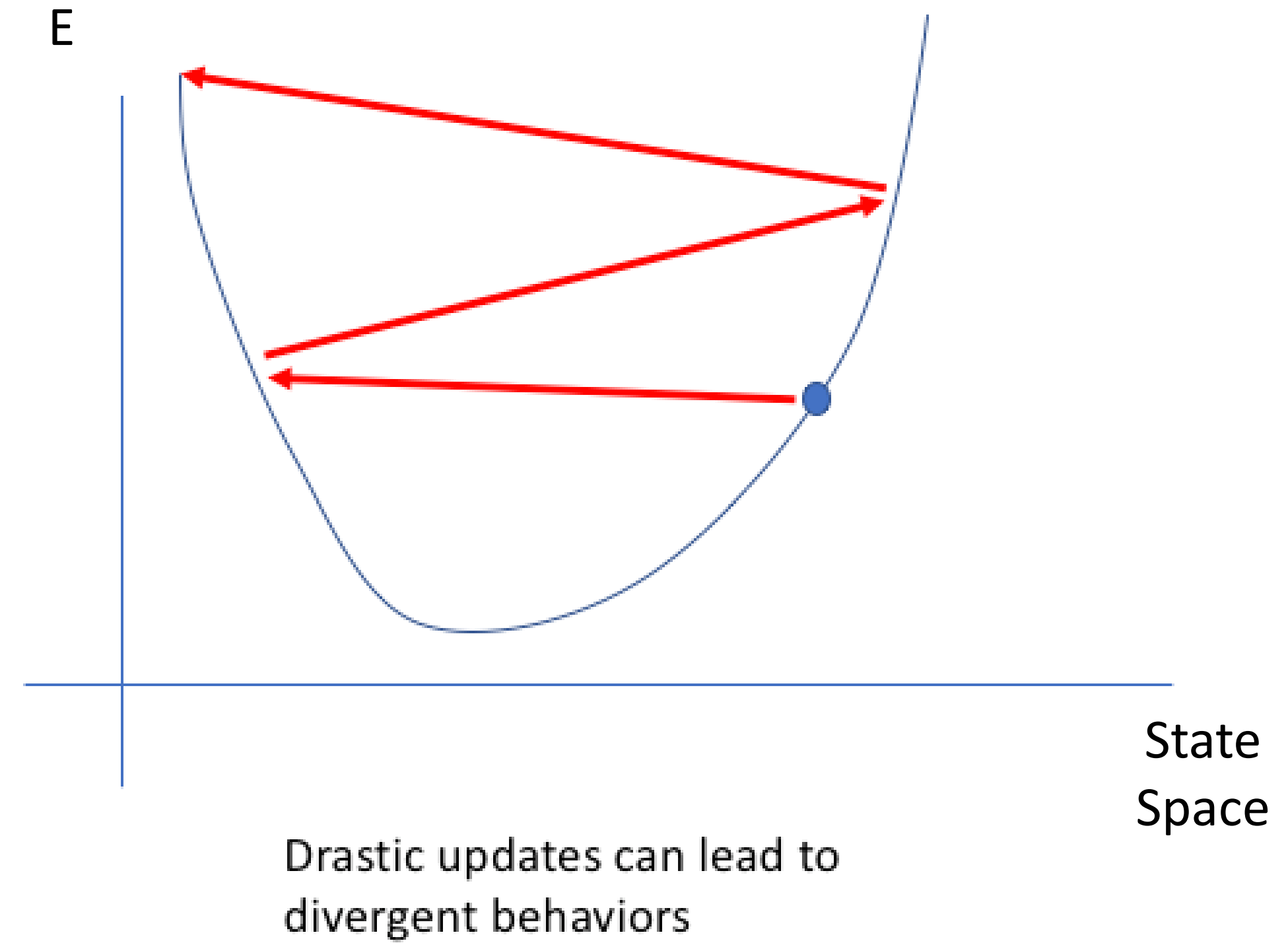
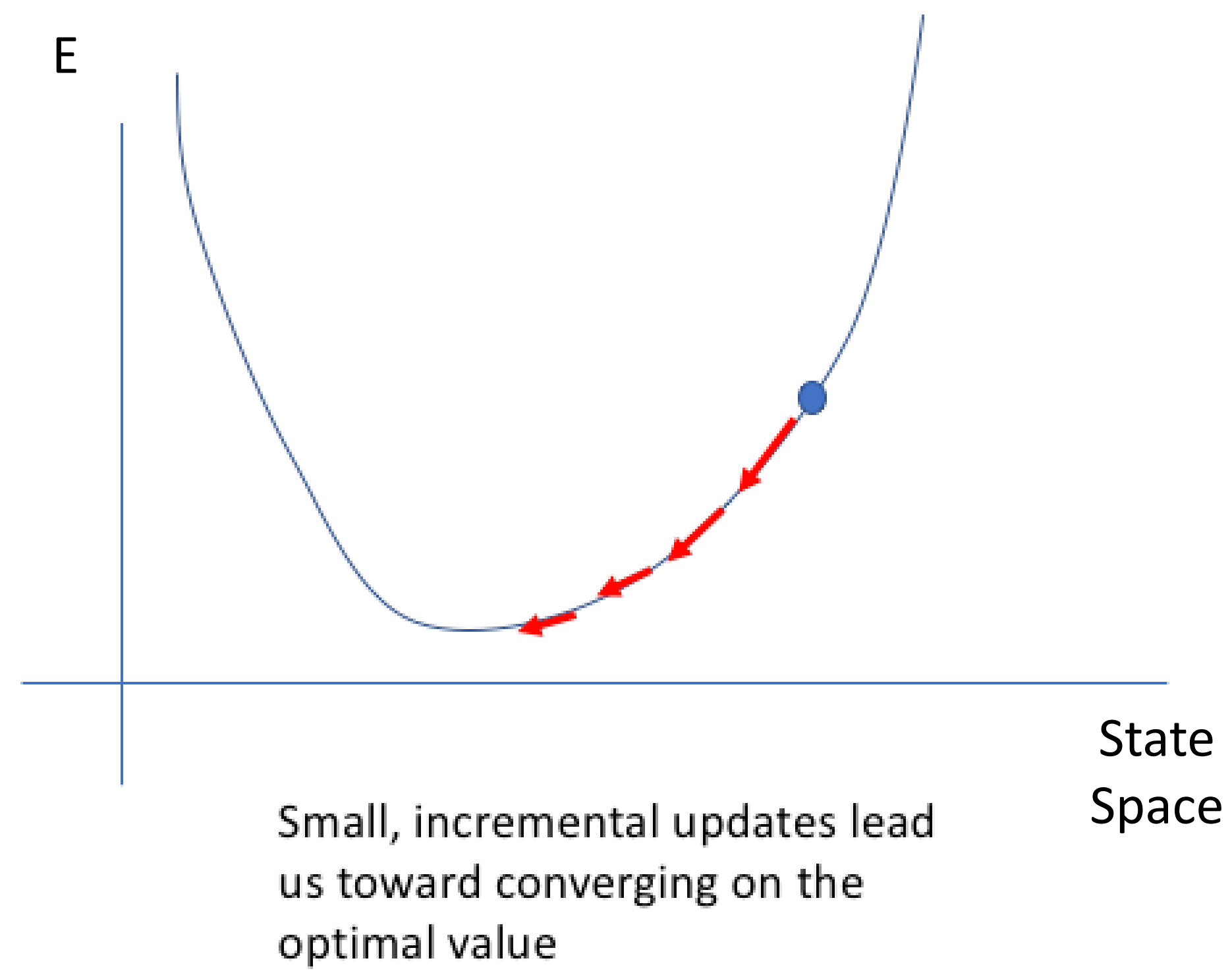
$$w \leftarrow w + \alpha \frac{\partial E}{\partial w}$$

Parameter α is the **learning rate**

- How the cost function affects the particular weight

Backpropagation

- Learning rate identify the step size.



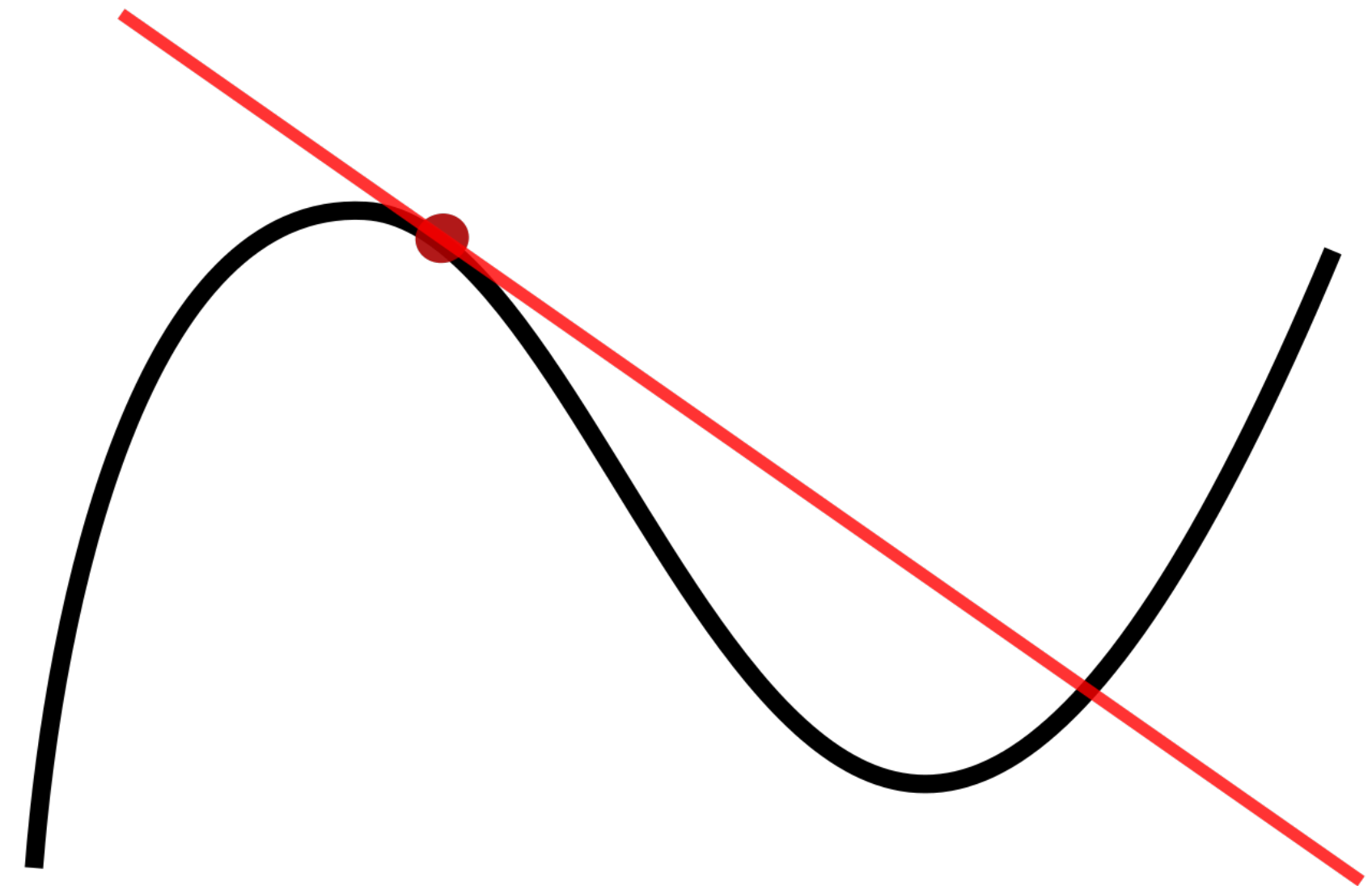
Backpropagation

The derivative of a function is the slope of the tangent at a point

$$y = f(x) = mx + b$$

$$m = \frac{\text{change in } y}{\text{change in } x} = \frac{\Delta y}{\Delta x}$$

Written $\frac{dy}{dx}$



Backpropagation

Partial derivative

Derivative of a function of several variables with respect to one of these variables

If $z = f(x, y, \dots)$

Derivative with respect to x is written: $\frac{\partial z}{\partial x}$

Neural Networks, Backpropagation

- We now describe how to compute the **partial derivatives of the loss function with respect to each weight**.
- For illustration, we consider a **two-layer neural network with sigmoid activation** at the hidden layer, as shown in the diagram.

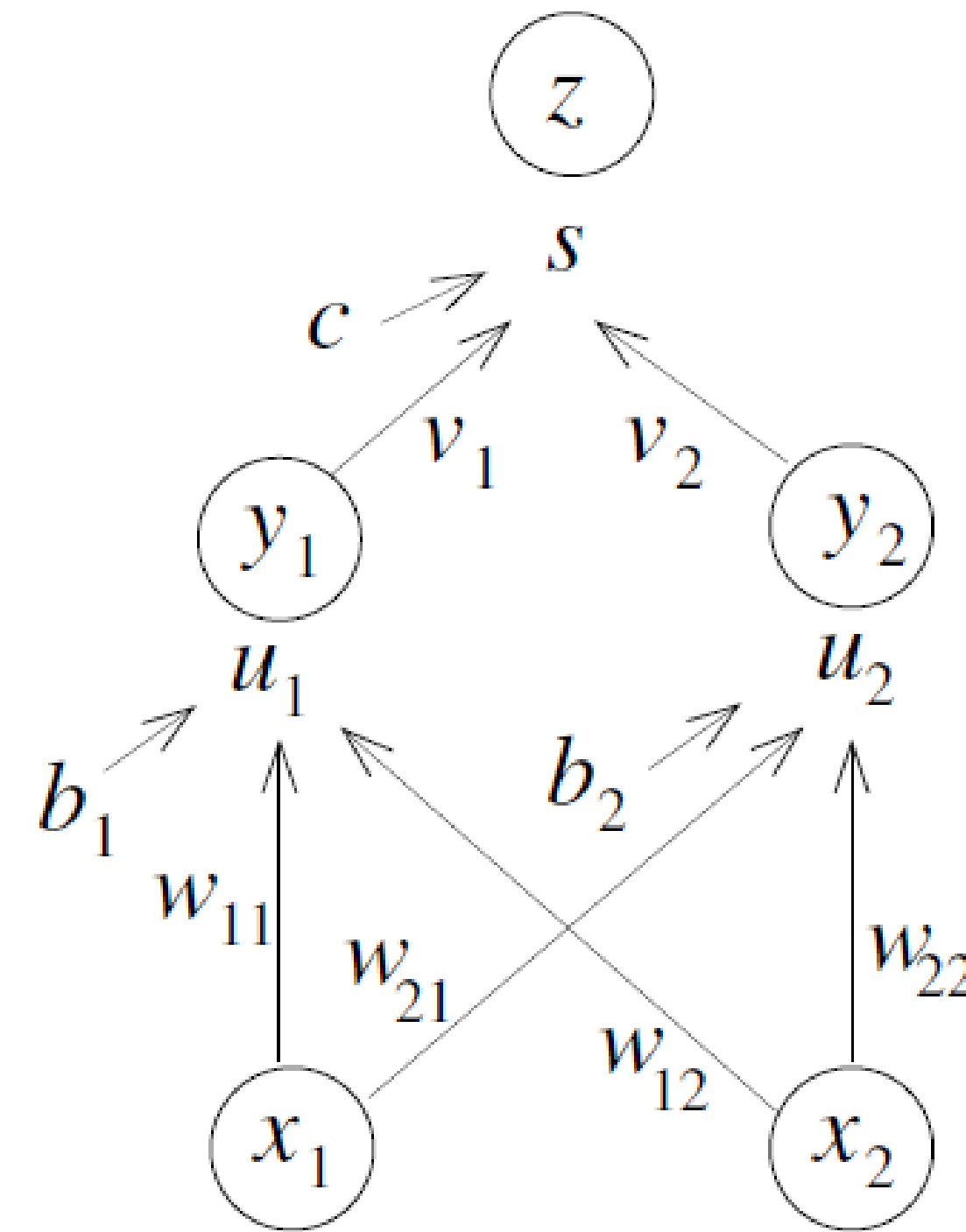
Here, x_i are the **inputs**, y_i are the **hidden units**, w_{ij} and v_i are the **weights**, and b_i and c are the **biases**.

$$u_1 = b_1 + w_{11}x_1 + w_{12}x_2$$

$$y_1 = g(u_1)$$

$$s = c + v_1y_1 + v_2y_2$$

$$z = g(s)$$



Neural Networks, Backpropagation

Chain Rule:

- If $y = y(u, v)$ where $u = u(x)$ and $v = v(x)$ then:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \frac{\partial u}{\partial x} + \frac{\partial y}{\partial v} \frac{\partial v}{\partial x}$$

- This principle can be used to compute the partial derivatives in an efficient manner. Note that the transfer function must be differentiable (usually sigmoid, or tanh).

$$\text{if } z(s) = \frac{1}{1 + e^{-s}}, z'(s) = z(1 - z)$$

$$\text{if } z(s) = \tanh(s), z'(s) = 1 - z^2$$

Neural Networks, Backpropagation

1. Forward pass:

$$u_1 = b_1 + w_{11}x_1 + w_{12}x_2$$

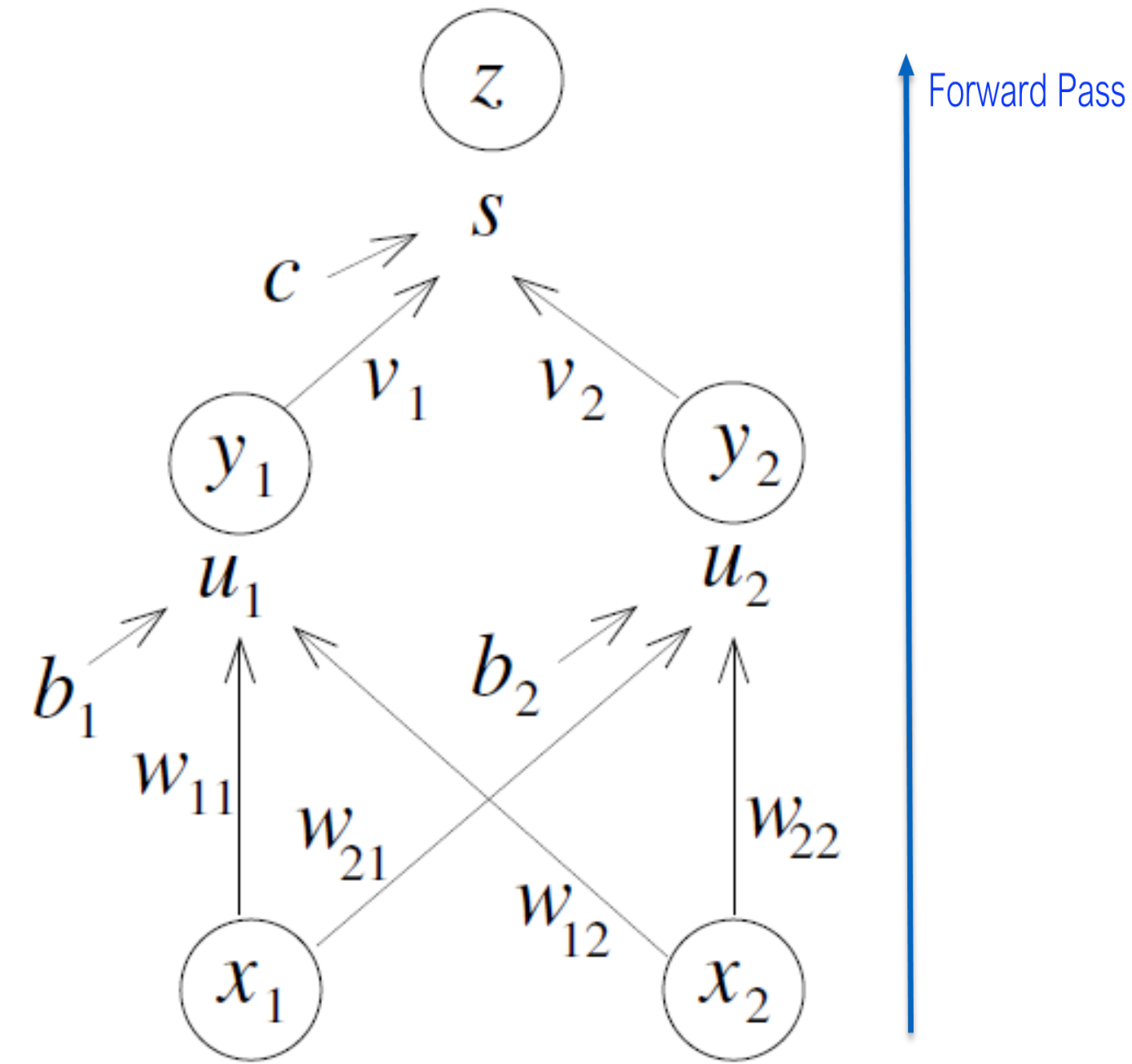
$$y_1 = g(u_1)$$

$$s = c + v_1y_1 + v_2y_2$$

$$z = g(s)$$

2. Calculate the error:

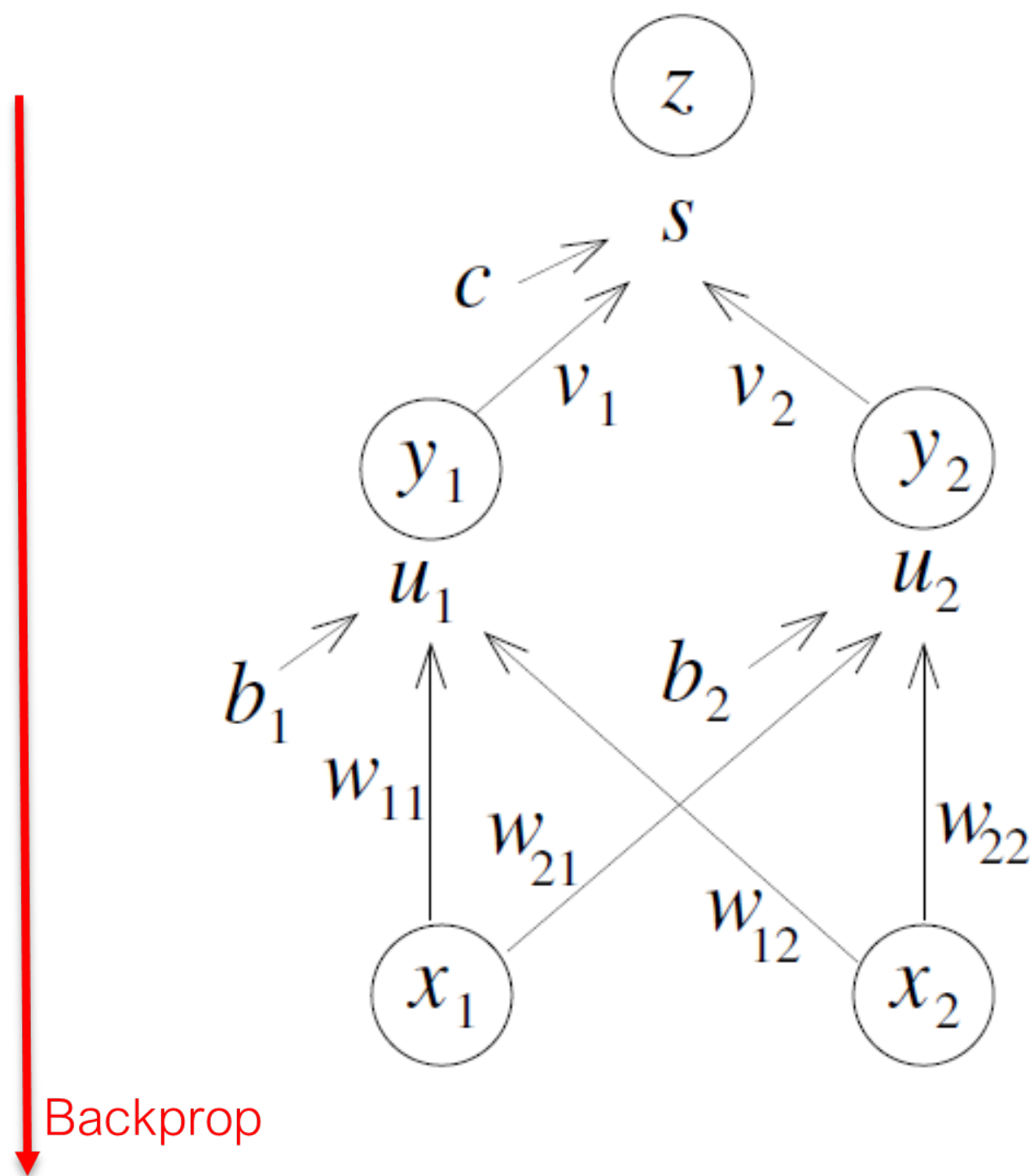
$$E = \frac{1}{2} \sum_i (z_i - t_i)^2$$



Neural Networks, Backpropagation

3. Backward pass:

Propagate errors back through the network to adjust weights ($E = \frac{1}{2} \sum_i (z_i - t_i)^2$).



Partial derivative:

$$\frac{\partial E}{\partial z} = z - t$$

$$\frac{dz}{ds} = g'(s) = z(1 - z)$$

$$\overline{\partial y_1} = v_1$$

$$\frac{\partial y_1}{\partial u_1} = y_1(1 - y_1)$$

Then:

$$\frac{\partial E}{\partial z} = (z - t)z (1 - z)$$

$$\frac{\partial s}{\partial E} = (z - t)z(1 - z)y_1$$

$$\frac{\partial E}{\partial u_1} = \frac{\partial E}{\partial z} \frac{\partial z}{\partial s} \frac{\partial s}{\partial y_1} \frac{\partial y_1}{\partial u_1} = (z - t)z(1 - z)v_1y_1(1 - y_1)$$

$$\frac{\partial E}{\partial w_{11}} = (z - t)z (1 - z)v_1 y_1 (1 - y_1) x_1$$

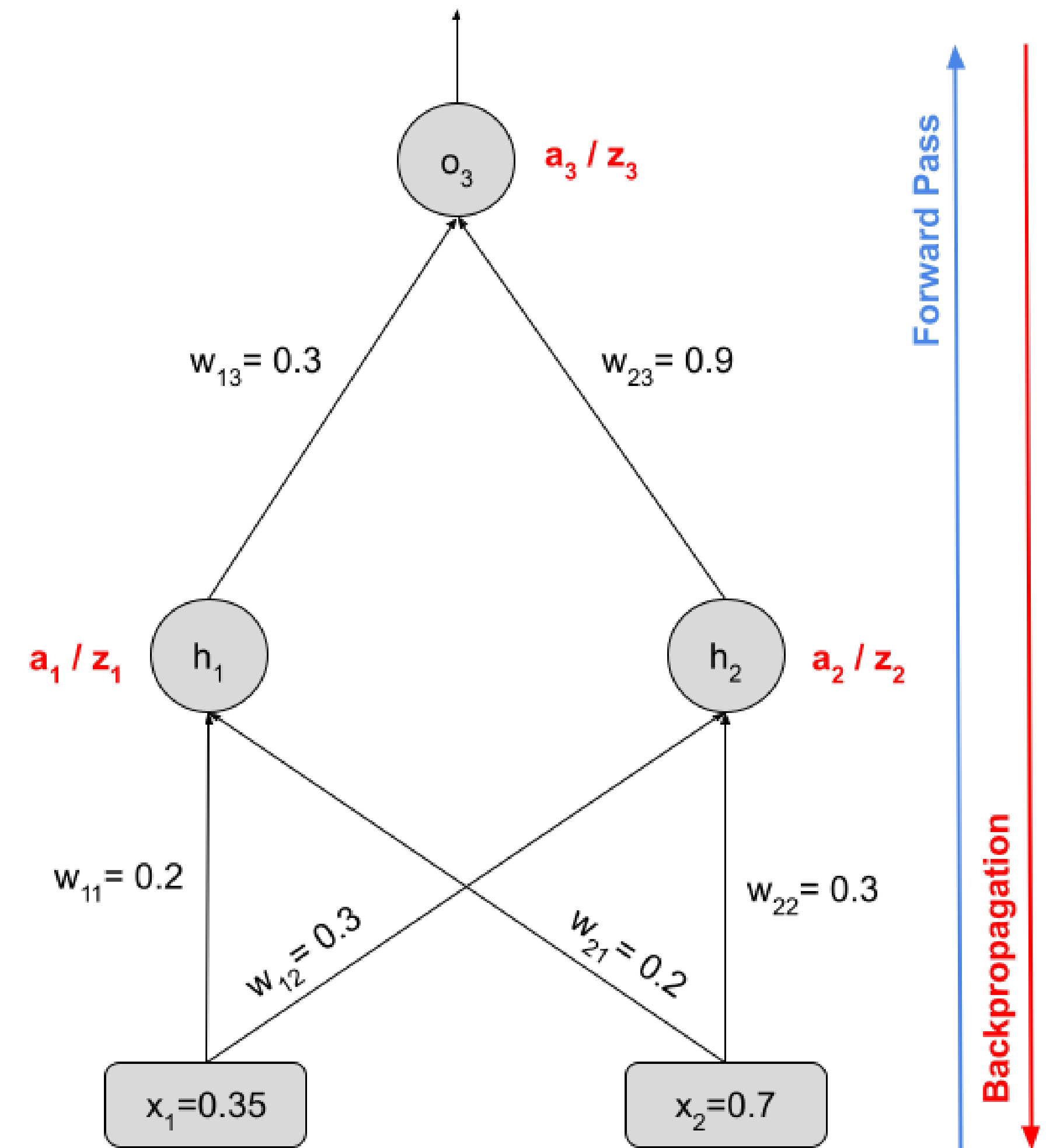
Neural Networks, Backpropagation

Example:

Consider the dataset $X = [0.35, 0.7]$ as the input and $y = [0.5]$ as the target output. We want to train the following neural network for 1 epoch and report the final weights. The activation function is the sigmoid function, and bias terms are set to 0. The error is defined as:

$$E = 1/2 (y_{\text{prediction}} - y_{\text{target}})^2$$

The initial values for the weights are shown in the figure.

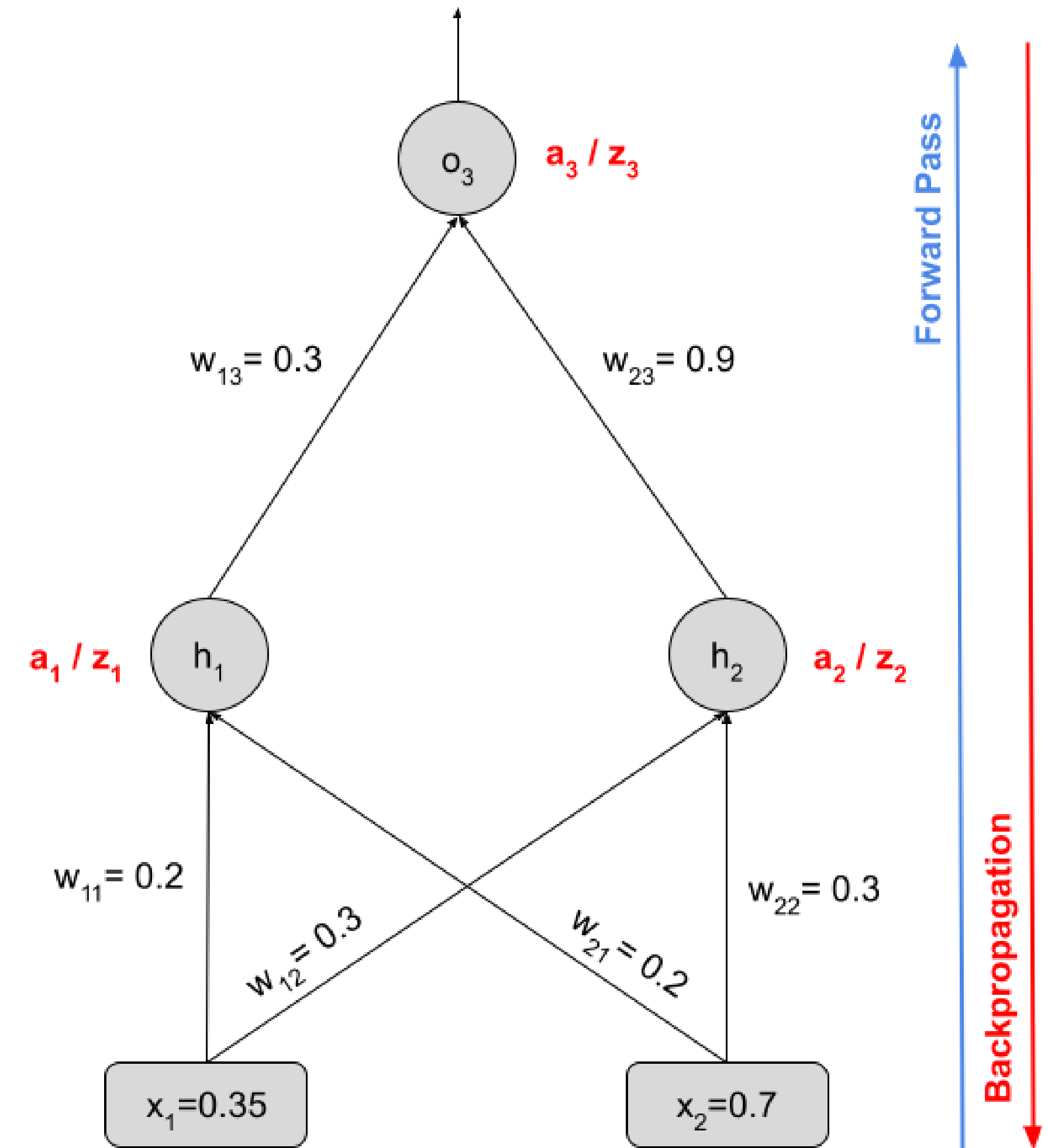


Neural Networks, Backpropagation

Step 1, Forward pass:

Computing the predicted output (y_3):

- $a_1 = b_1 + w_{11}x_1 + w_{21}x_2 = 0 + (0.2 \times 0.35) + (0.2 \times 0.7) = 0.21$
- $z_1 = g(a_1) = \frac{1}{1 + e^{-0.21}} = \mathbf{0.55}$
- $a_2 = b_2 + w_{12}x_1 + w_{22}x_2 = 0 + (0.3 \times 0.35) + (0.3 \times 0.7) = 0.315$
- $z_2 = g(a_2) = \frac{1}{1 + e^{-0.315}} = \mathbf{0.58}$
- $a_3 = b_3 + w_{13}z_1 + w_{23}z_2 = 0 + (0.3 \times 0.55) + (0.9 \times 0.58) = 0.687$
- $z_3 = g(a_3) = \frac{1}{1 + e^{-0.687}} = \mathbf{0.67}$

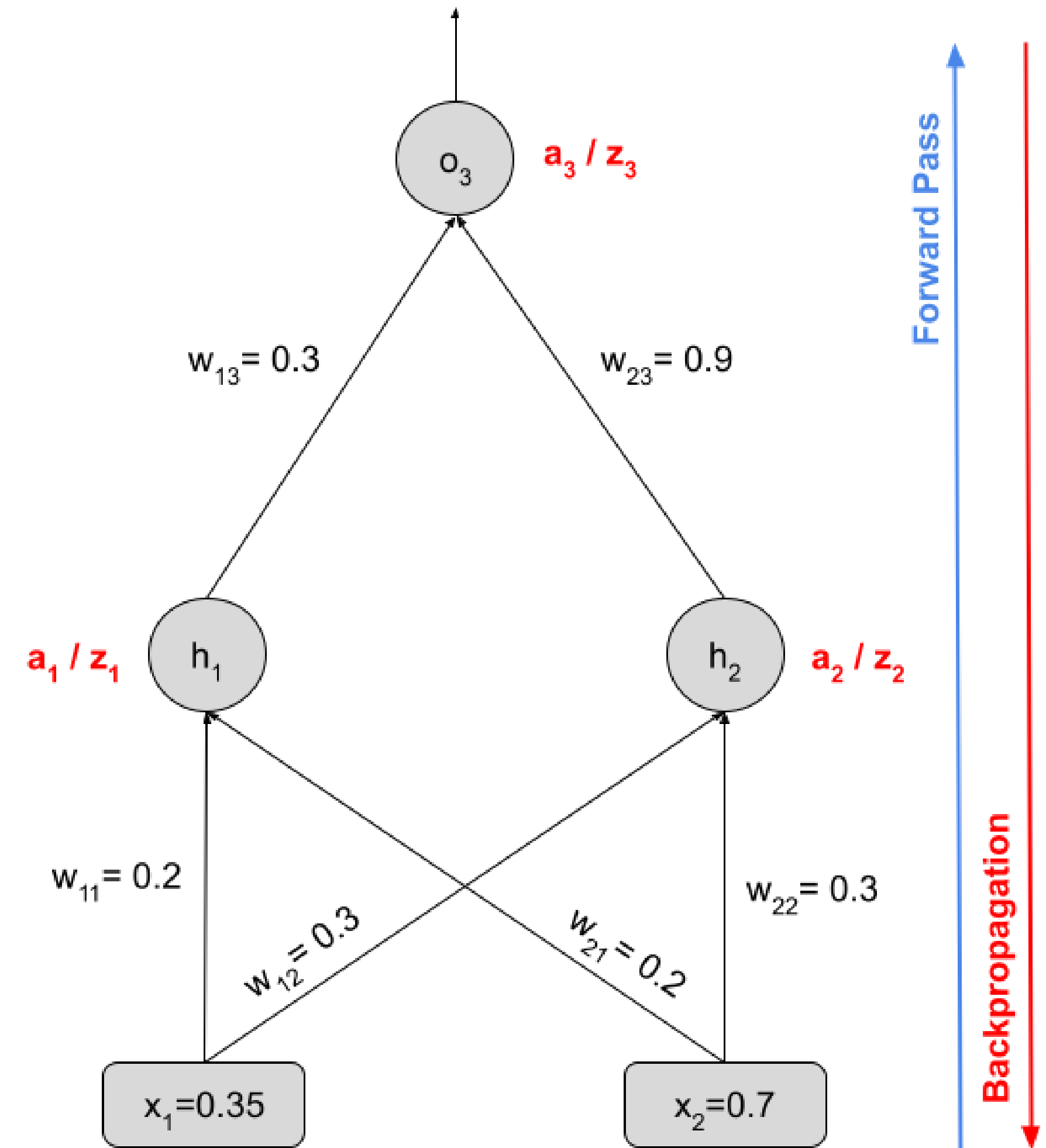


Neural Networks, Backpropagation

Step 2, Calculate the error:

$$E = 1/2 (y_{\text{prediction}} - y_{\text{target}})^2$$

$$\text{Error} = \frac{1}{2} (0.67 - 0.5)^2 = 0.014$$



Neural Networks, Backpropagation

Step 3, Backpropagation:

$$w_{ij}^{\text{new_epoch}} \leftarrow w_{ij}^{\text{old_epoch}} - \eta \frac{\partial E}{\partial w_{ij}}$$

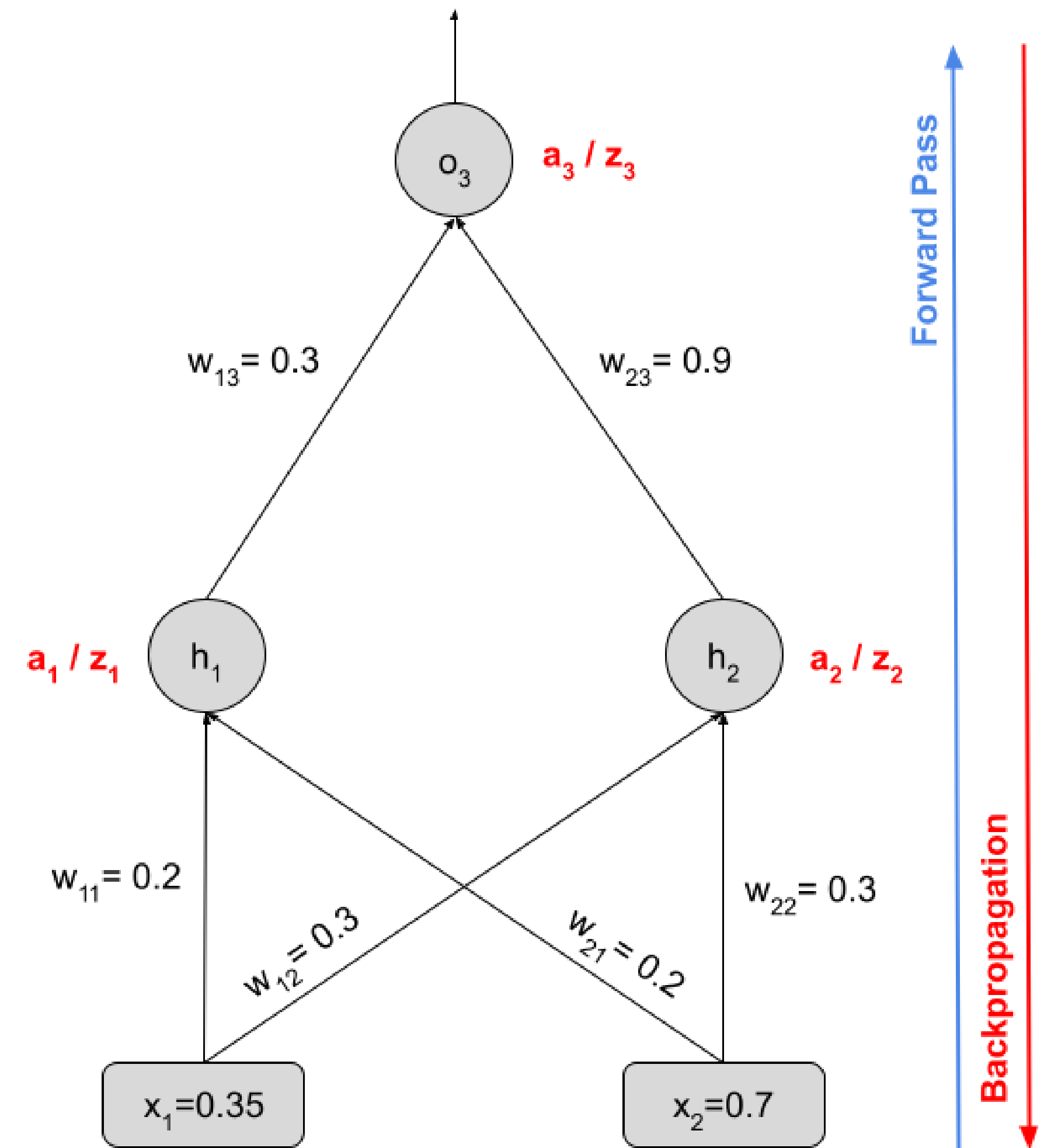
$$\frac{\partial E}{\partial z_3} = z_3 - y_{\text{target}} = 0.67 - 0.5 = 0.17$$

$$\frac{\partial z_3}{\partial a_3} = z_3(1 - z_3) = 0.67(1 - 0.67) = 0.221$$

$$\frac{\partial a_3}{\partial w_{23}} = z_2 = 0.58$$

$$\frac{\partial E}{\partial w_{23}} = \frac{\partial E}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_3} \cdot \frac{\partial a_3}{\partial w_{23}} = 0.17 \times 0.22 \times 0.58 = 0.021$$

$$w_{23}^{\text{new_epoch}} = 0.9 - 1 \times (0.021) = 0.879$$



Neural Networks, Backpropagation

Step 3, Backpropagation:

$$w_{ij}^{\text{new_epoch}} \leftarrow w_{ij}^{\text{old_epoch}} - \eta \frac{\partial E}{\partial w_{ij}}$$

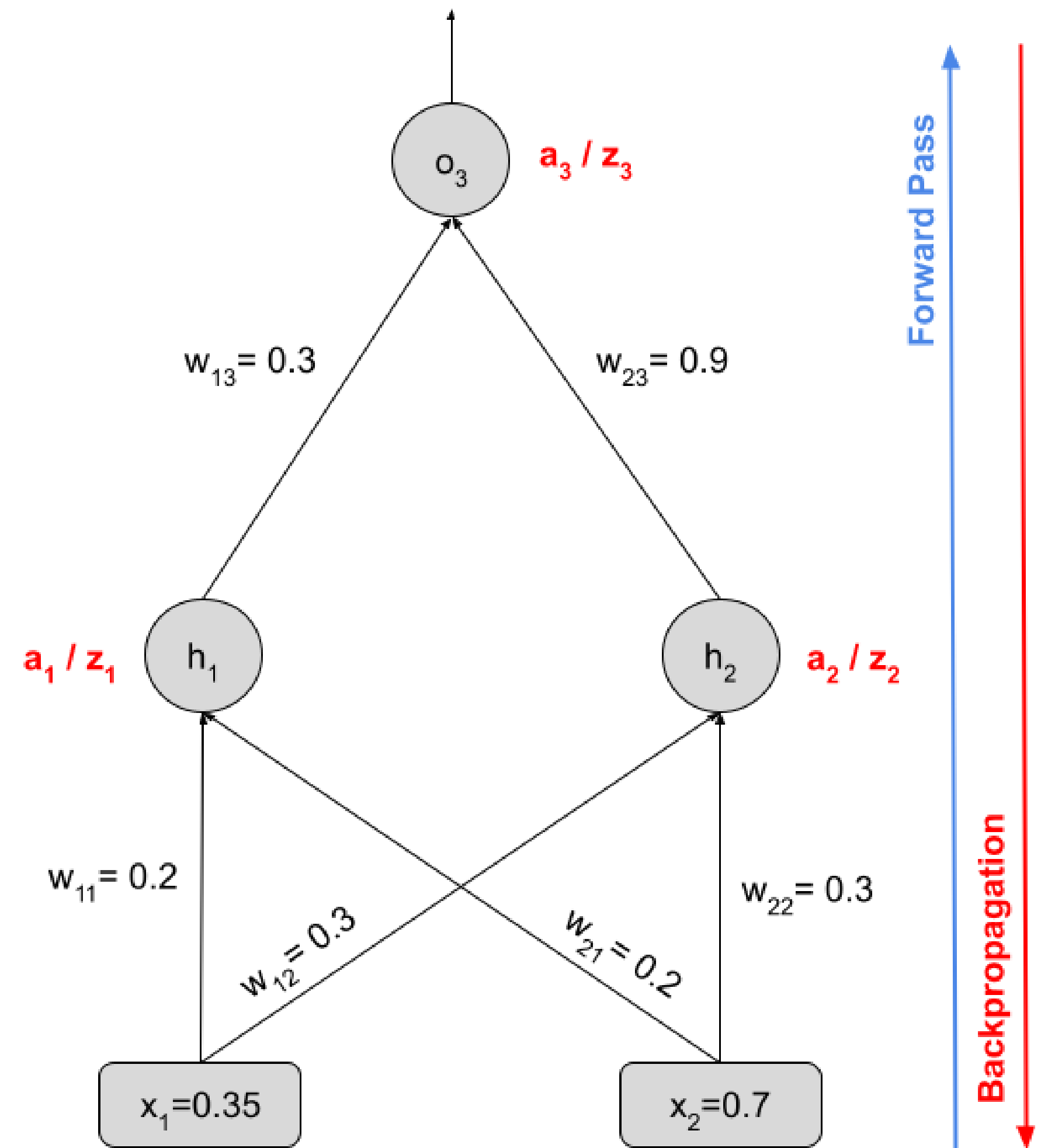
$$\frac{\partial E}{\partial z_3} = z_3 - y_{\text{target}} = 0.67 - 0.5 = 0.17$$

$$\frac{\partial z_3}{\partial a_3} = z_3(1 - z_3) = 0.67(1 - 0.67) = 0.221$$

$$\frac{\partial a_3}{\partial w_{13}} = z_1 = 0.55$$

$$\frac{\partial E}{\partial w_{13}} = \frac{\partial E}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_3} \cdot \frac{\partial a_3}{\partial w_{13}} = 0.17 \times 0.22 \times 0.55 = 0.020$$

$$w_{13}^{\text{new_epoch}} = 0.3 - 1 \times (0.020) = 0.28$$



Neural Networks, Backpropagation

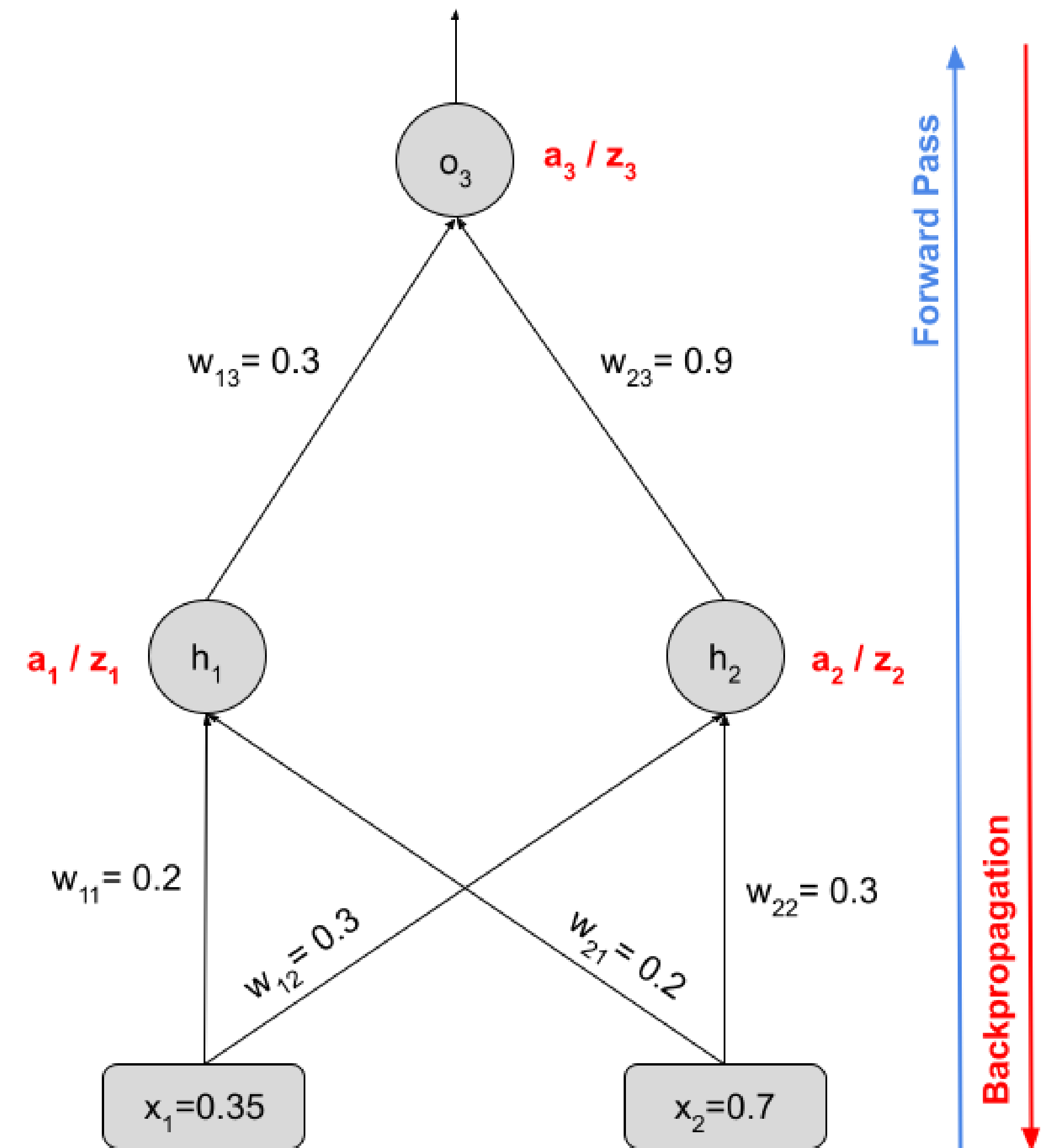
Step 3, Backpropagation:

$$w_{ij}^{\text{new_epoch}} \leftarrow w_{ij}^{\text{old_epoch}} - \eta \frac{\partial E}{\partial w_{ij}}$$

$$\frac{\partial E}{\partial w_{11}} = \frac{\partial E}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_1} \cdot \frac{\partial z_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_{11}}$$
$$\frac{\partial a_3}{\partial z_1} = w_{13} ; \frac{\partial z_1}{\partial a_1} = z_1(1 - z_1) ; \frac{\partial a_1}{\partial w_{11}} = x_1$$

$$\frac{\partial E}{\partial w_{11}} = 0.17 \times 0.221 \times 0.3 \times 0.2475 \times 0.35 = 0.0009$$

$$w_{11}^{\text{new_epoch}} = 0.2 - 1 \times (0.0009) = 0.1991$$



Neural Networks, Backpropagation

Step 3, Backpropagation:

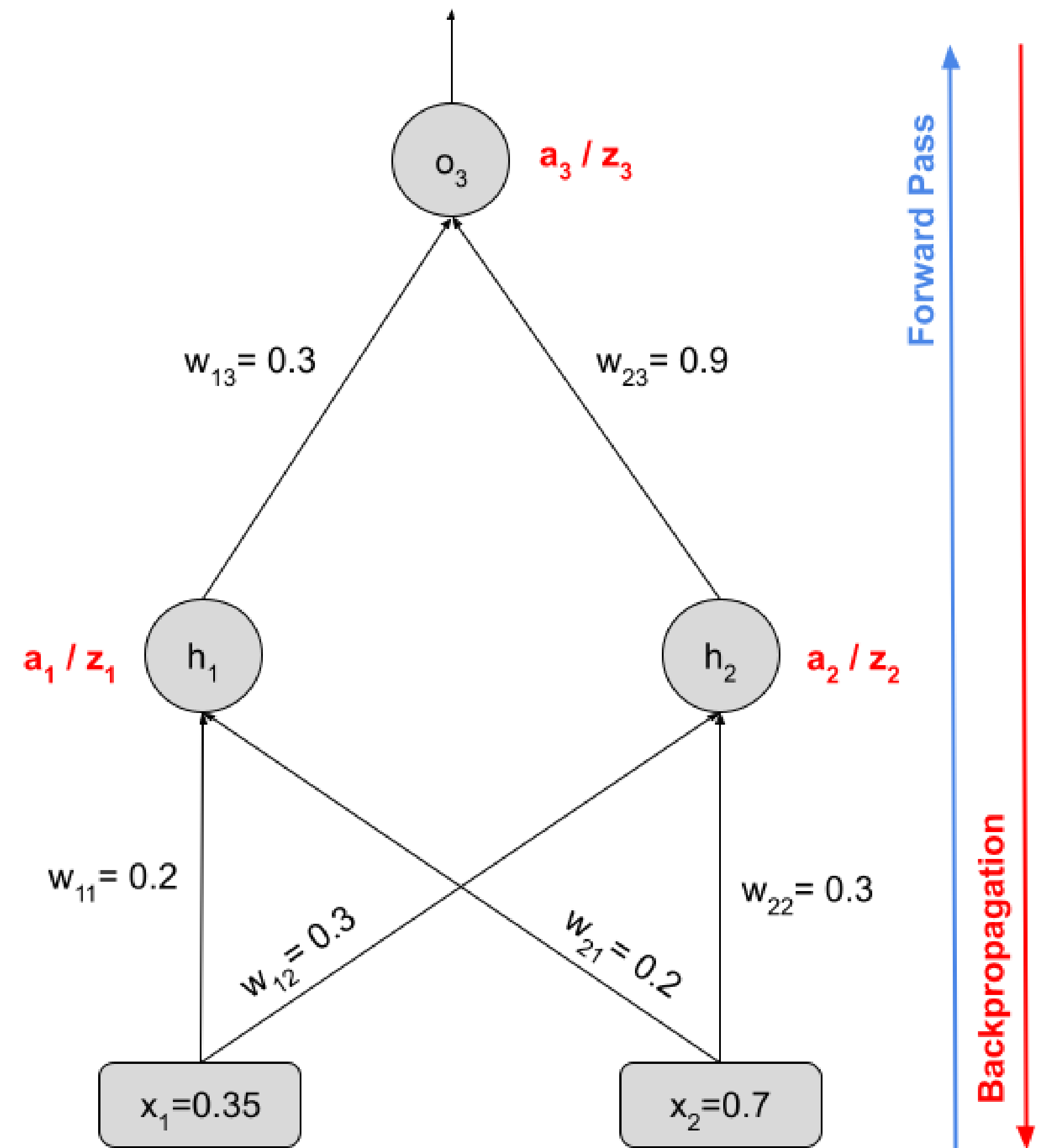
$$w_{ij}^{\text{new_epoch}} \leftarrow w_{ij}^{\text{old_epoch}} - \eta \frac{\partial E}{\partial w_{ij}}$$

$$\frac{\partial E}{\partial w_{21}} = \frac{\partial E}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_1} \cdot \frac{\partial z_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_{21}}$$

$$\frac{\partial a_1}{\partial w_{21}} = x_2$$

$$\frac{\partial E}{\partial w_{11}} = 0.17 \times 0.2221 \times 0.3 \times 0.2475 \times 0.7 = 0.002$$

$$w_{21}^{\text{new_epoch}} = 0.2 - 1 \times (0.002) = 0.198$$



Neural Networks, Backpropagation

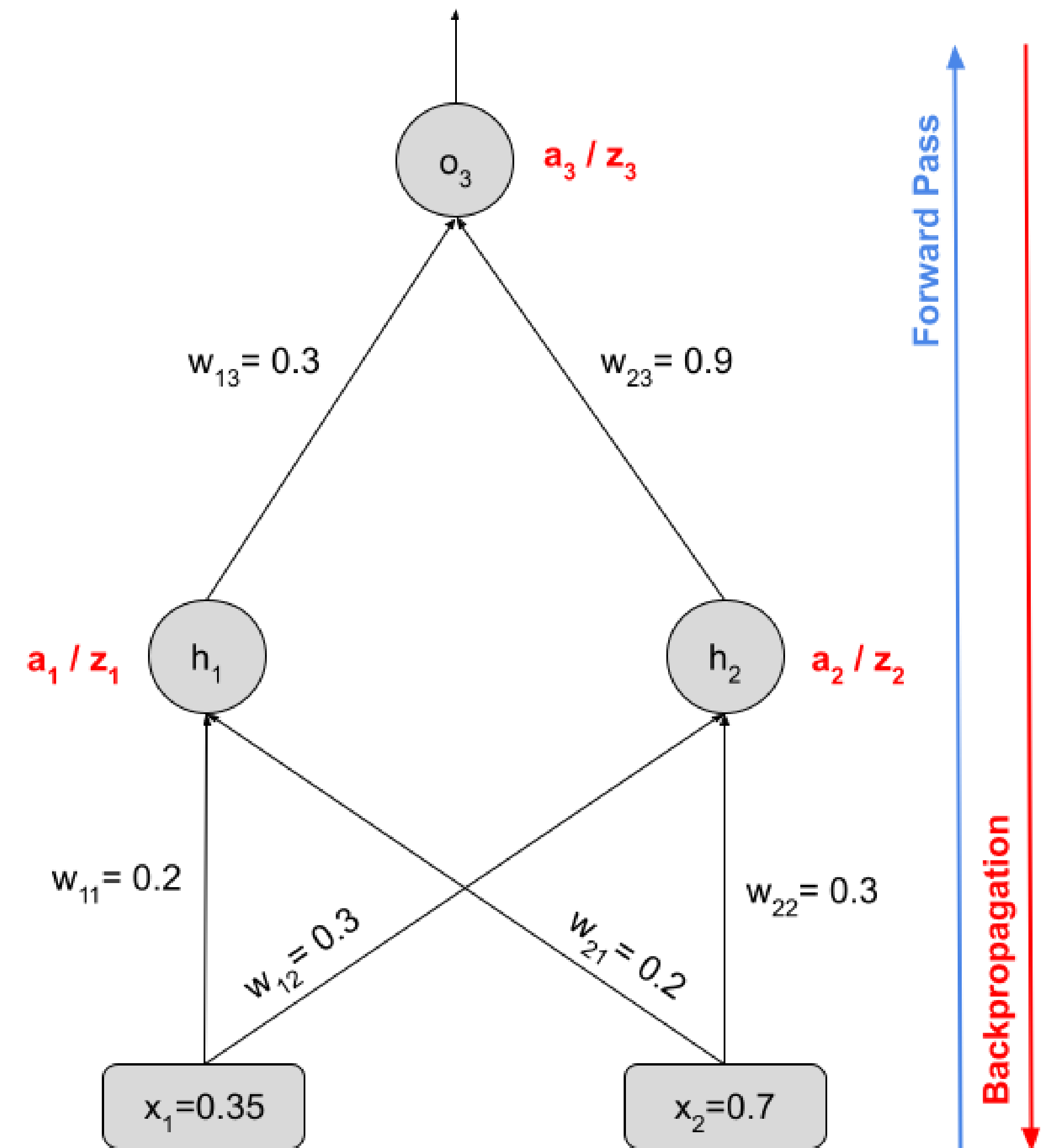
Step 3, Backpropagation:

$$w_{ij}^{\text{new_epoch}} \leftarrow w_{ij}^{\text{old_epoch}} - \eta \frac{\partial E}{\partial w_{ij}}$$

$$\frac{\partial E}{\partial w_{12}} = \frac{\partial E}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial w_{12}}$$
$$\frac{\partial a_3}{\partial z_2} = w_{23} ; \frac{\partial z_2}{\partial a_2} = z_2(1 - z_2) ; \frac{\partial a_2}{\partial w_{12}} = x_1$$

$$\frac{\partial E}{\partial w_{12}} = 0.17 \times 0.2221 \times 0.9 \times 0.2436 \times 0.35 = 0.00288$$

$$w_{12}^{\text{new_epoch}} = 0.3 - 1 \times (0.00288) = 0.297$$



Neural Networks, Backpropagation

Step 3, Backpropagation:

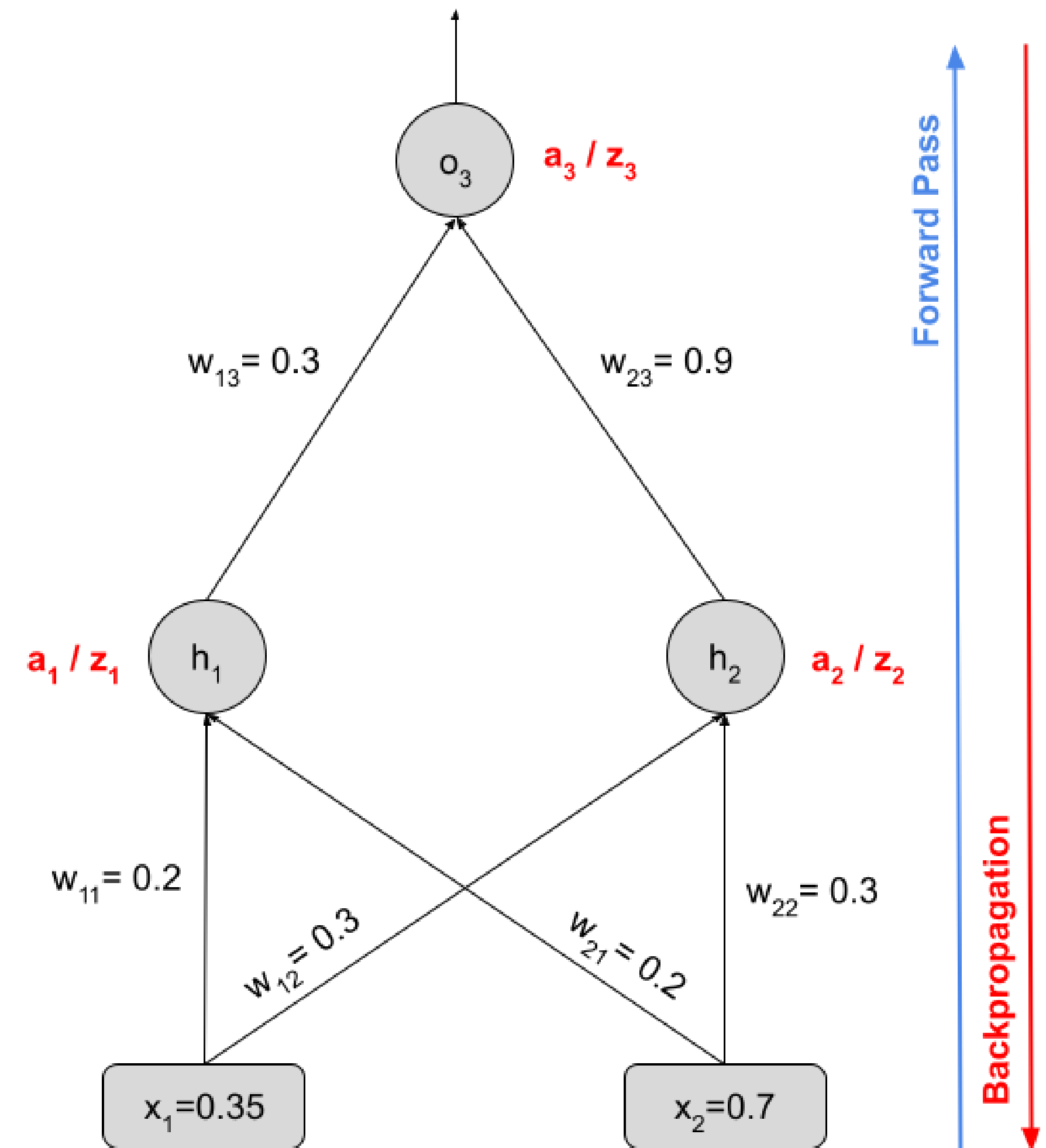
$$w_{ij}^{\text{new_epoch}} \leftarrow w_{ij}^{\text{old_epoch}} - \eta \frac{\partial E}{\partial w_{ij}}$$

$$\frac{\partial E}{\partial w_{22}} = \frac{\partial E}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial w_{22}}$$

$$\frac{\partial a_2}{\partial w_{22}} = x_2$$

$$\frac{\partial E}{\partial w_{22}} = 0.17 \times 0.2221 \times 0.9 \times 0.2436 \times 0.7 = 0.0057$$

$$w_{22}^{\text{new_epoch}} = 0.3 - 1 \times (0.0057) = 0.294$$



Neural Networks, Backpropagation

Conclusion:

$$y_{target} < y_{prediction} \quad (0.5 < 0.67)$$

$$w_{11}^{old_epoch} = 0.2 \rightarrow w_{11}^{new_epoch} = 0.1991$$

$$w_{21}^{old_epoch} = 0.2 \rightarrow w_{21}^{new_epoch} = 0.198$$

$$w_{12}^{old_epoch} = 0.3 \rightarrow w_{12}^{new_epoch} = 0.297$$

$$w_{22}^{old_epoch} = 0.3 \rightarrow w_{22}^{new_epoch} = 0.294$$

$$w_{13}^{old_epoch} = 0.3 \rightarrow w_{13}^{new_epoch} = 0.28$$

$$w_{23}^{old_epoch} = 0.9 \rightarrow w_{23}^{new_epoch} = 0.87$$

- Since our prediction is greater than the target value, we need to **decrease the weight** in order to reduce the prediction in the next iterations.
- As we propagate back to the weights in the earlier layers, the amount of adjustment (decrement) becomes **smaller**. For example, the change in w_{13} is more significant than the change in w_{11} . Why is that?

Lecture Overview

- Motivation
- Biological and artificial neurons
- Single-layer perceptron
- Multi-layer perceptron
- **Neural network design**
- Neural network architectures

Step 1: Exhaustive Analysis of the System

- It should determine the number and type of input variables and model output, reducing the number of variables.
- Is it really necessary to use a neural model? Why not use any other existing classic model (e.g., phenomenological)?
 - Neural Network: Best second solution.
- If a neural model is used, do we have available data representing properly the system to be modelled? Do we have enough?

Step 2: Preprocessing

- **Data:** a neural network is a black-box model (a.k.a. empirical model) for interpolation (**never extrapolation**); therefore, they greatly depend on quality and quantity of data available.
- **Quality:** related to the degree to which the available data represents the function being approximated. **Ideal:** to obtain them by following a properly designed survey/experimental plan.
- **Quantity:** It is extremely important because only an adequate amount of data will allow us to correctly identify the parameters (weights) of our neural model.
 - If the quantity of data is small, we **cannot expect** to develop a complex neural model.

Step 2: Preprocessing

- Visual examination of the data.
 - Detect and, if possible, eliminate outliers, empty values, etc.
 - It might also help to detect correlations between variables.
- **Normalisation of variables:** It is necessary when variables with different units and, therefore, potentially different magnitudes are involved. Sometimes, the magnitudes can differ by several orders of magnitude.
 - $X_n = (X - X_{\min}) / (X_{\max} - X_{\min}); X_n \in [0, 1]$
 - $X_n = 2 * (X - X_{\min}) / (X_{\max} - X_{\min}) - 1; X_n \in [-1, 1]$
- It is necessary to perform the corresponding denormalization at the output stage.

Step 3: Design of the Neural Model

- Input and output neurons depend on the previous analysis of the system.
- But, what about the number of neurons N_h in the hidden layer?
- **Rule of thumb:** N_h should lead to a number of parameters (weights) N_w that:
 - $N_w < (\text{Number of samples}) / 10$
- The number of weights N_w of an MLP, with N_i neurons in its input layer, a hidden layer with N_h neurons, and N_o neurons in the output layer is:
 - $N_w = (N_i + 1) * N_h + (N_h + 1) * N_o$

Step 3: Design of the Neural Model

- An MLP with 3 inputs, 4 units in its hidden layer, and 2 outputs, has a number of parameters:
 - $N_w = (3+1)*4+(4+1)*2 = 26$
- Then, at least 260 samples are required to train the network weights.

Step 3: Design of the Neural Model

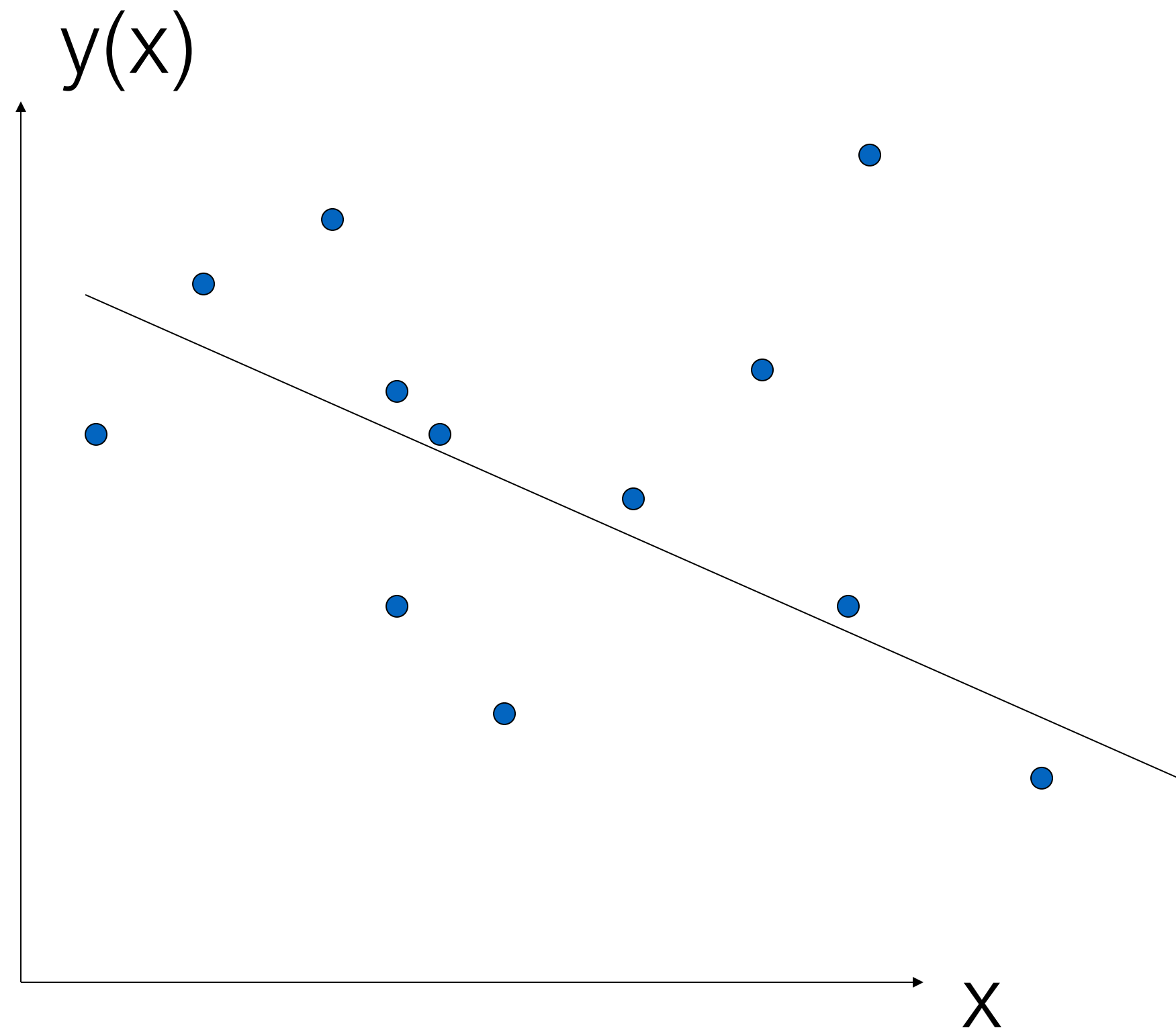
- In MLPs is demonstrated that using one hidden layer with a proper number of neurons is sufficient to approximate any non-linear function with an arbitrary precision degree.
- **Activation functions:** A usual criterion is to use sigmoid functions or ReLUs in the hidden layer and linear functions in the output. However, sigmoids or softmax can also be used in the output.

Step 4: Training

- Training a neural network is a hard process due to the complexity of the error function solution space, which can have numerous local minima, saddle (minimax) points, etc.
- There are two main problems that can arise during training:
 - Underfitting (high bias)
 - Overfitting (high variance)
- The latter might affect the network's ability to generalise.

Step 4: Training

Training bias

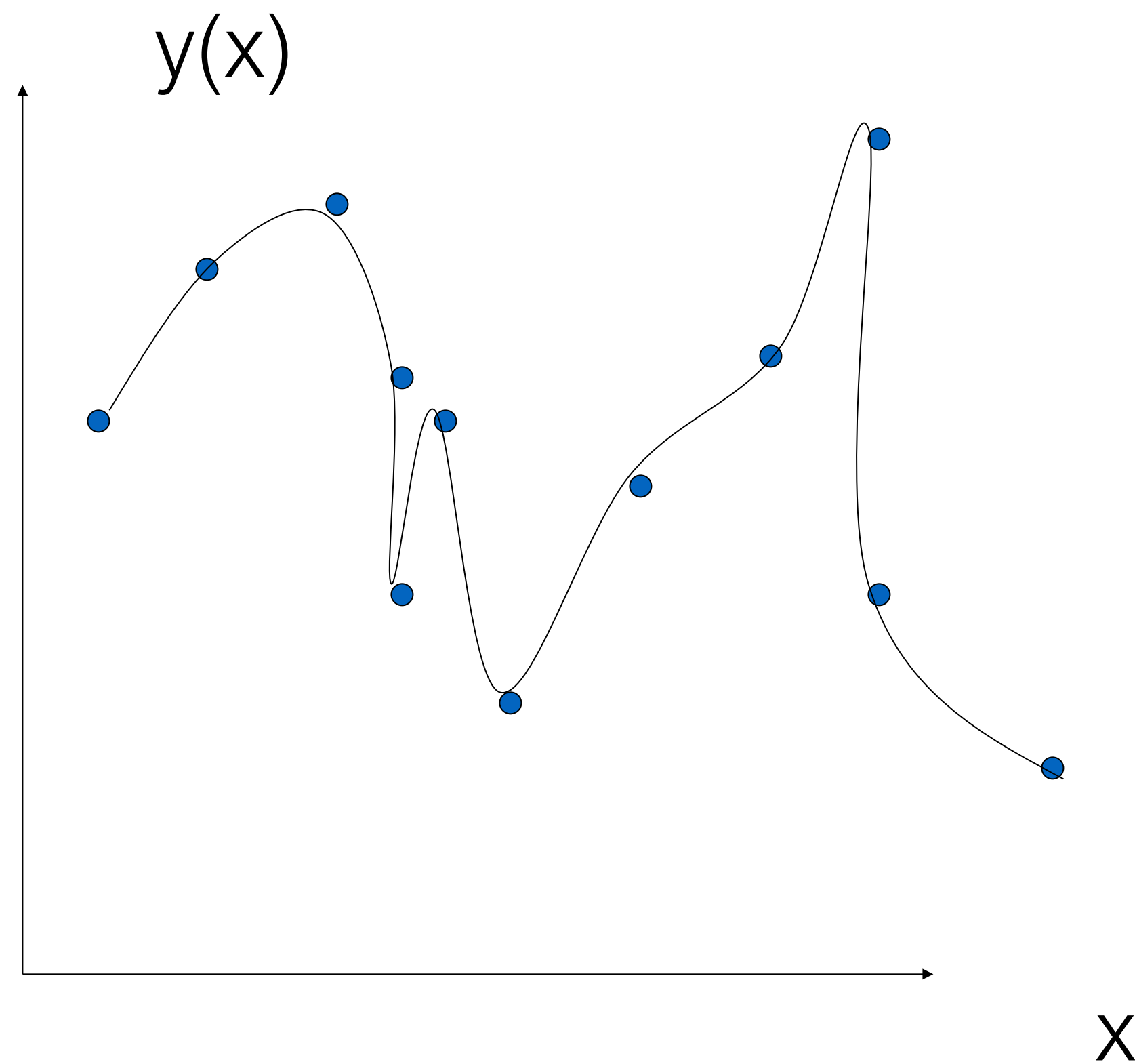


Step 4: Training

- Bias refers to the **systematic error** introduced by approximating a real-world problem (which may be complex) with a simpler model (high bias causes underfitting).
- To decrease bias:
 - Increase (prudently) the number of neurons in the hidden layer.
 - Aim to reach a better local minimum by conducting a sufficient number of different training processes, starting from randomly chosen initial weights (20 or more attempts).

Step 4: Training

High variance problem
(overparameterization and
overfitting)

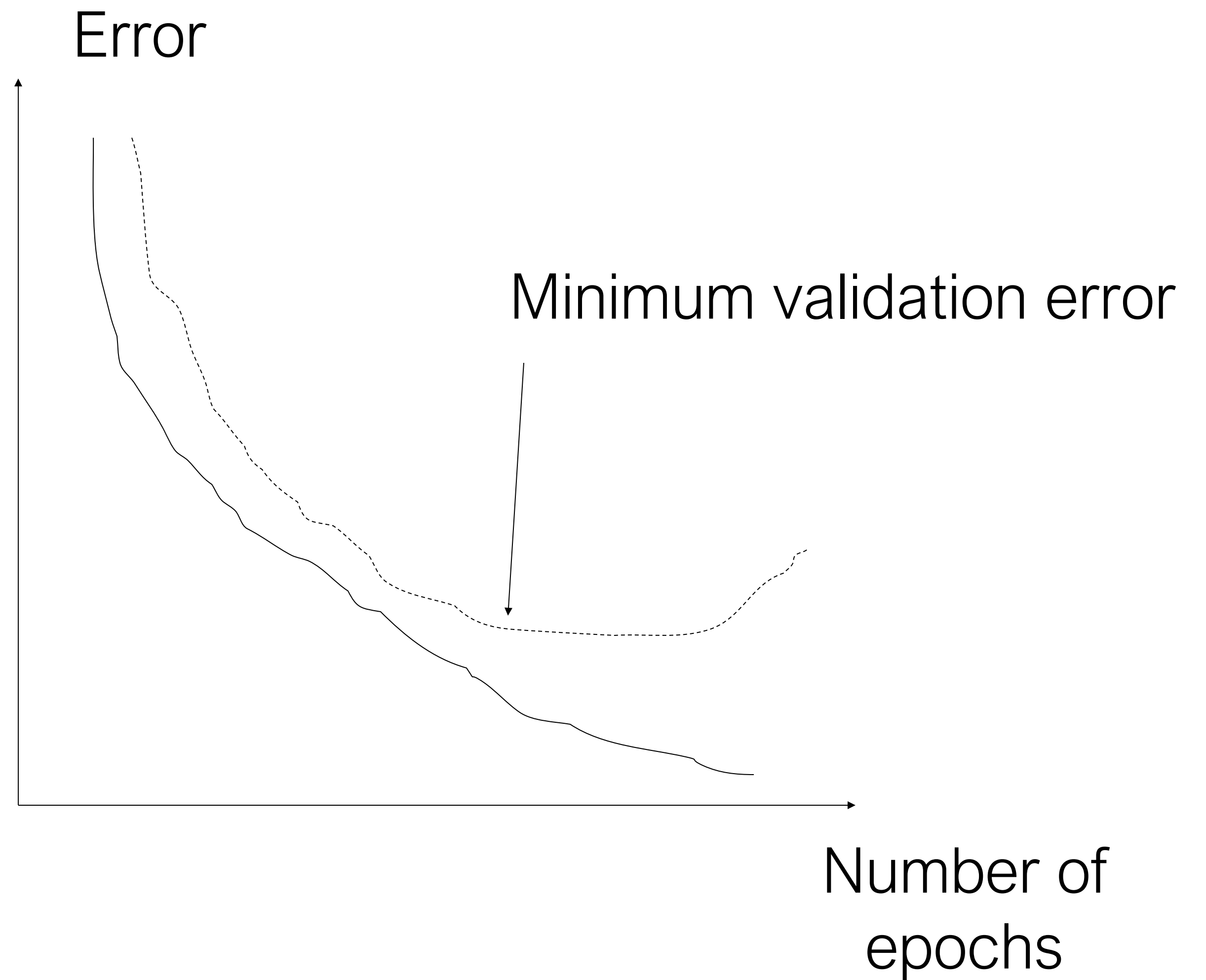


Step 4: Training

- To avoid overfitting problem, the dataset is often divided into **Training, Validation, and Test (generalisation) sets.**
- The best is to visualise the error function simultaneously on both sets.
- Characteristics of the training and validation sets:
 - Both sets should be large enough, and data should be representative on both sets.

Step 4: Training

Error function for
training set (-) and
validation set (---)

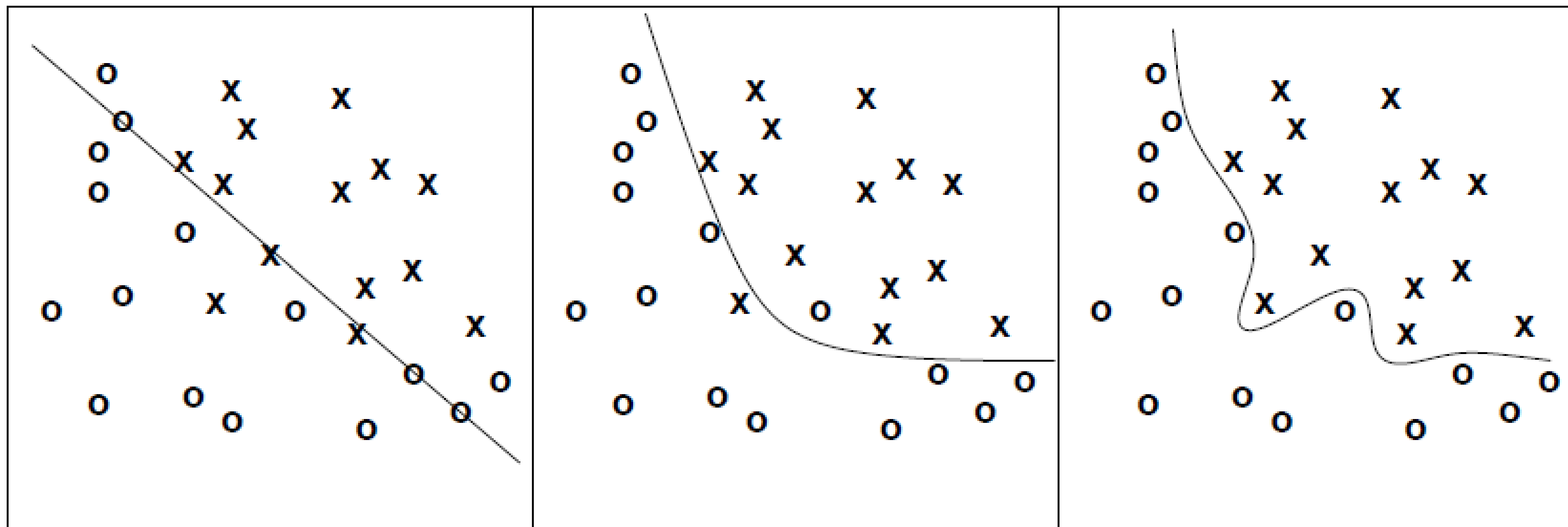


Step 4: Training

- **Cross-validation:** Different neural network models are developed using the available data, splitting the training and validation sets in different ways. The model that achieves the minimum error on the validation set is chosen.
- **Additional training aspects:**
 - Weight initialisation.
 - Online or batch learning.
 - Adjust the parameters, e.g., learning rate and epochs to suit the particular task.

Step 5: Testing and Evaluation

- Ockham's Razor: “The most likely hypothesis is the simplest one consistent with the data”
- In the middle curve, two training items near the boundary are misclassified. This is acceptable since the errors may be caused by data noise, which often arises from small unmodeled factors such as measurement errors, friction, or air resistance.

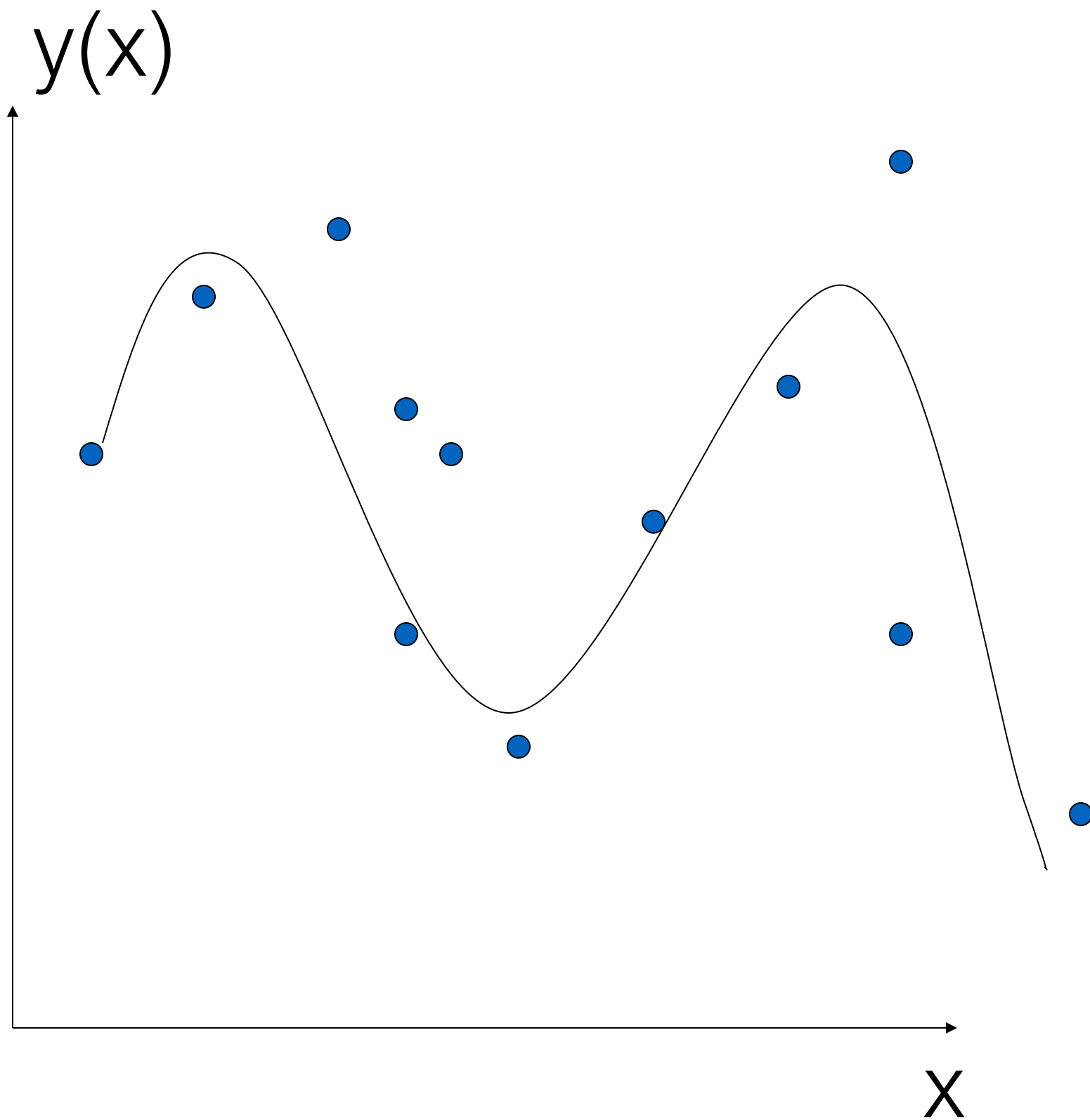


Step 5: Testing and Evaluation

- To test the generalisation capability of the network, that is, its performance on a different (never seen) set of data, a small (but representative) third set might be reserved, [the test set](#).
- This set should also be representative of the phenomenon being modelled as the previous sets (training and validation).

Step 5: Testing and Evaluation

Approximation of the
underlying function



Step 5: Testing and Evaluation

Some error indexes used to test results:

$$IA = 1 - \frac{\sum_{i=1}^n (o_i - p_i)^2}{\sum_{i=1}^n (|o_i'| + |p_i'|)^2}$$

$$RMS = \sqrt{\frac{\sum_{i=1}^n (o_i - p_i)^2}{\sum_{i=1}^n o_i^2}}$$

$$RSD = \sqrt{\frac{\sum_{i=1}^n (o_i - p_i)^2}{N}}$$

o_i : Observed values.

p_i : Predicted values

N : Number of data samples.

o_m : Observations' mean value.

$$o_i' = o_i - o_m$$

$$p_i' = p_i - o_m$$

Step 5: Testing and Evaluation

- To test the generalisation capability of the network, performance is evaluated on [the test set](#).
- Some common loss functions:

Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Cross-Entropy (Binary Cross-Entropy (for 2 classes)):

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - \hat{y}_i) \log(1 - \hat{y}_i)]$$

Lecture Overview

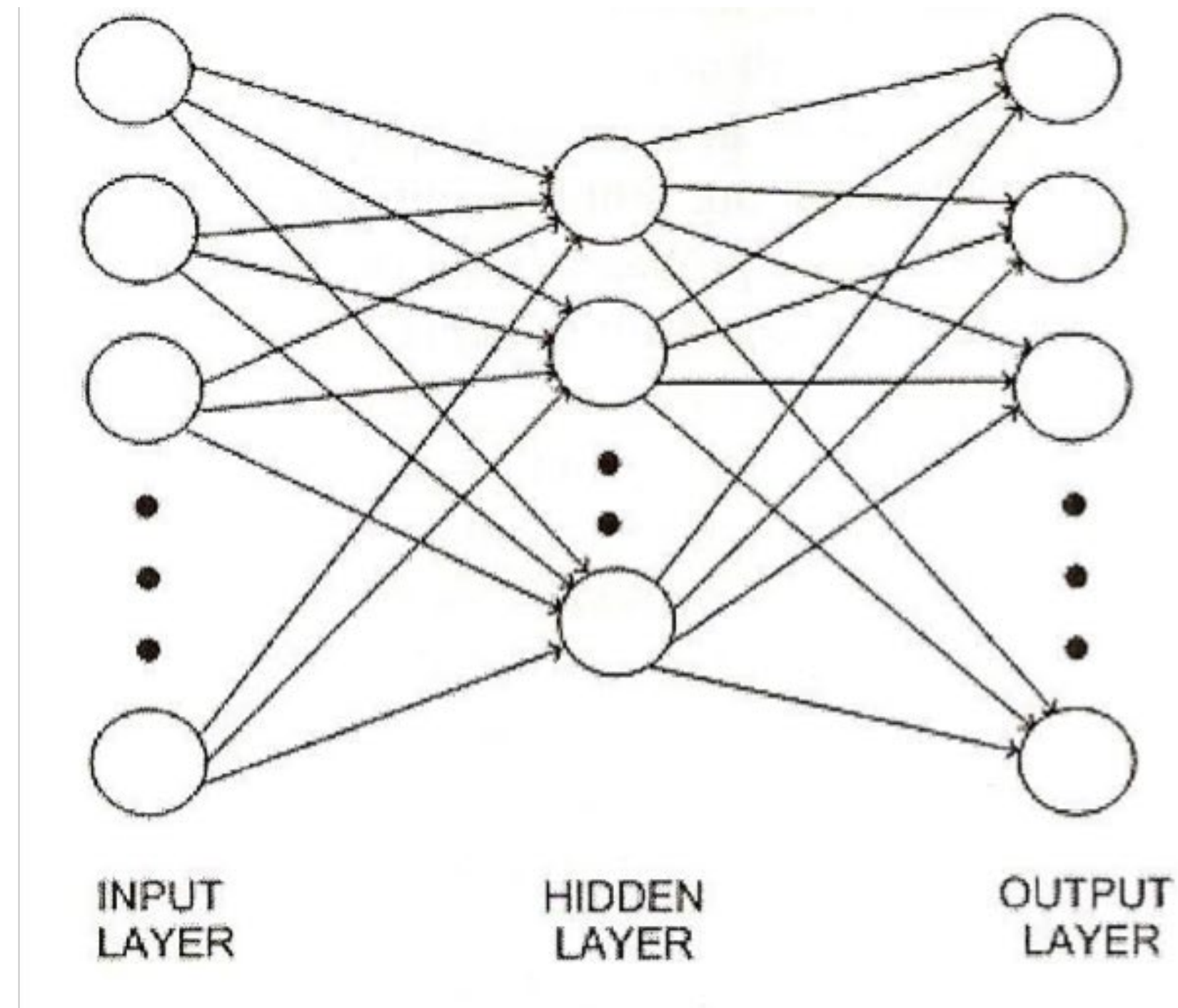
- Motivation
- Biological and artificial neurons
- Single-layer perceptron
- Multi-layer perceptron
- Neural network design
- **Neural network architectures**

Neural Network Architectures

- Two main network structures

1. Feed-Forward Network

2. Recurrent Network



Neural Network Architectures

Feed-forward network has connections only in one direction:

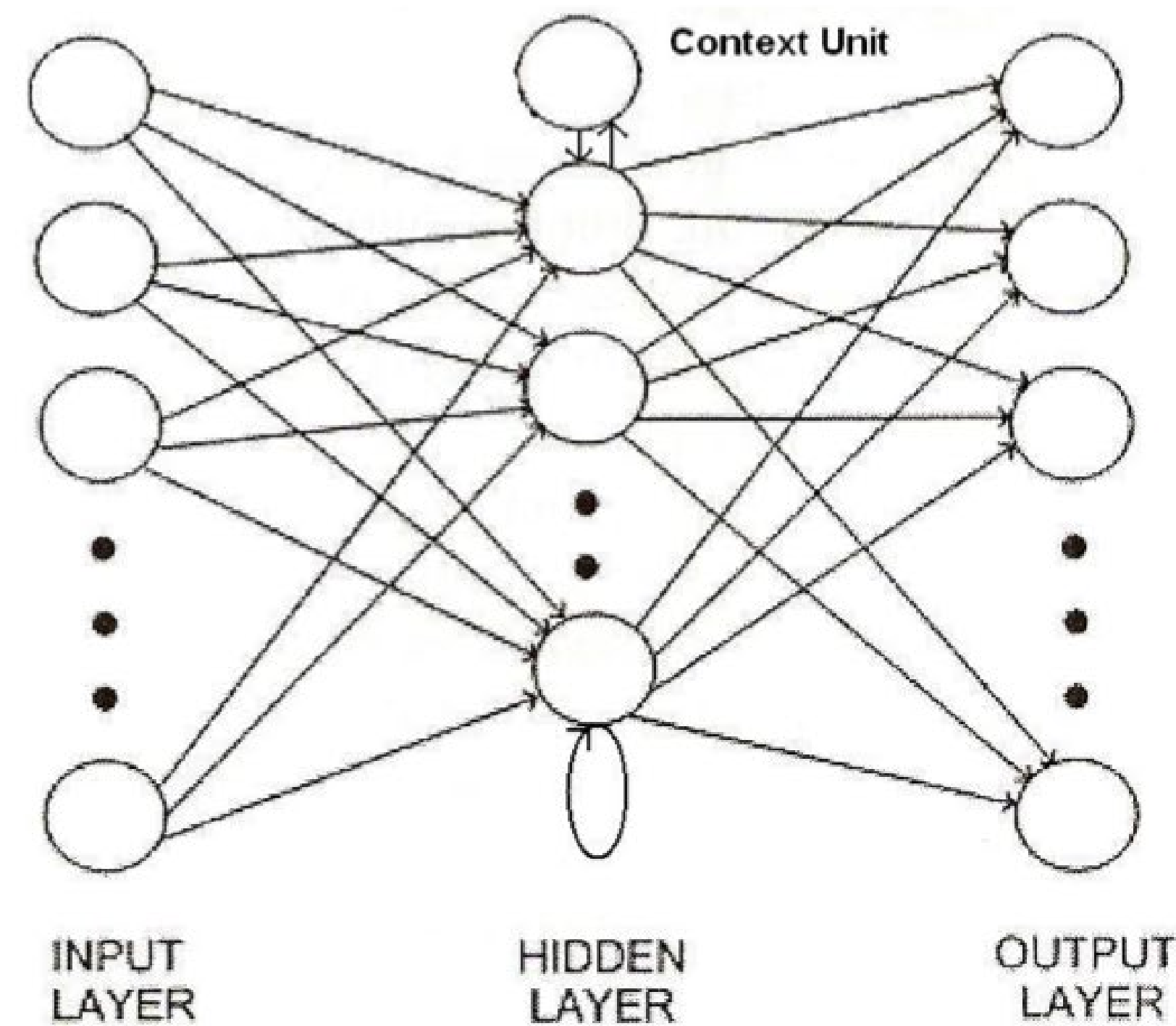
- Every node receives input from “upstream” nodes; delivers output to “downstream” nodes.
 - No loops.
- Represents a function of its current input.
 - It has no internal state other than the weights themselves.

Neural Network Architectures

- Two main network structures

1. Feed-Forward Network

2. Recurrent Network



Neural Network Architectures

Recurrent network feeds outputs back into its own inputs:

- Activation levels of network form a dynamical system.
 - It may reach a stable state or exhibit oscillations or even chaotic behaviour.
- Response of network to an input depends on its initial state.
 - This may depend on previous inputs.
- Can support short-term memory.

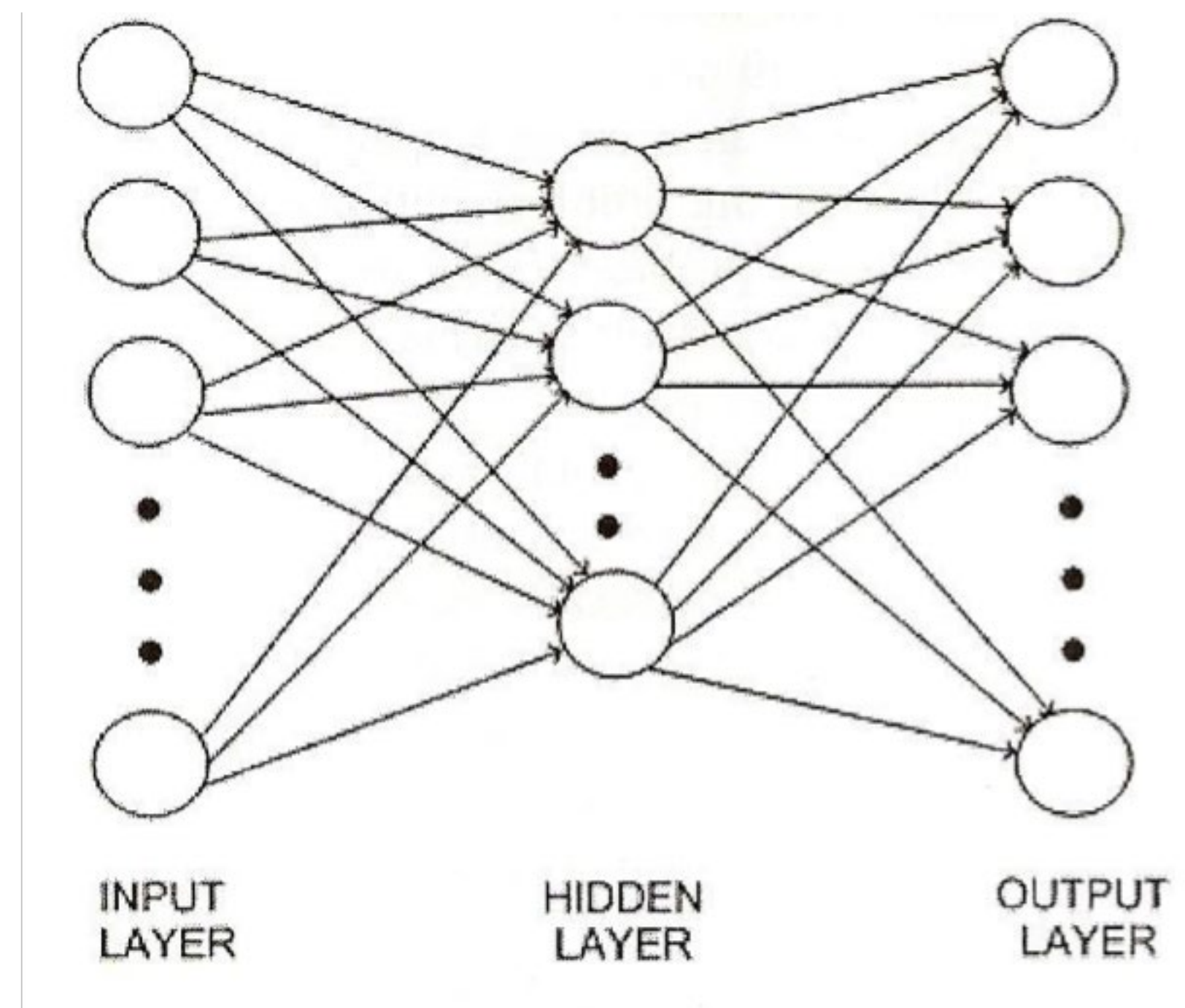
Deep Learning Architectures

- Multiple layers form a hierarchical model, known as **deep learning**.
 - **Convolutional neural networks** are specialised for vision tasks.
 - **Recurrent neural networks** are used for time series.
- A typical real-world network can have 10 to hundreds of layers with hundreds of millions of weights:
 - It can take hours, days, or months to learn on machines with thousands of cores.

Deep Learning Architectures

The **depth** of a model is defined as the total number of layers with learnable parameters, excluding the input layer.

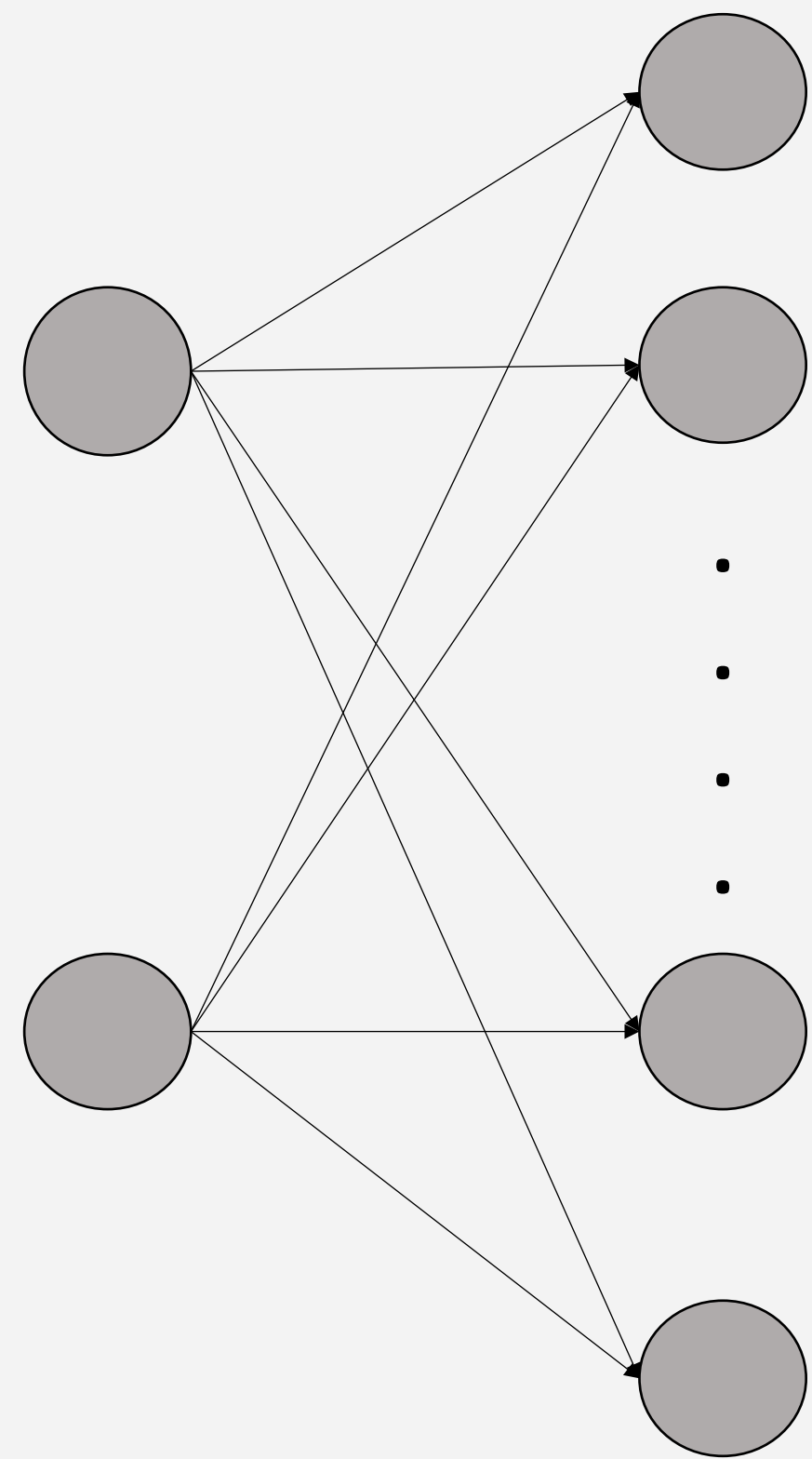
- Deeper networks can typically approximate more complex functions, while shallower models have fewer layers and are limited to approximating simpler functions.



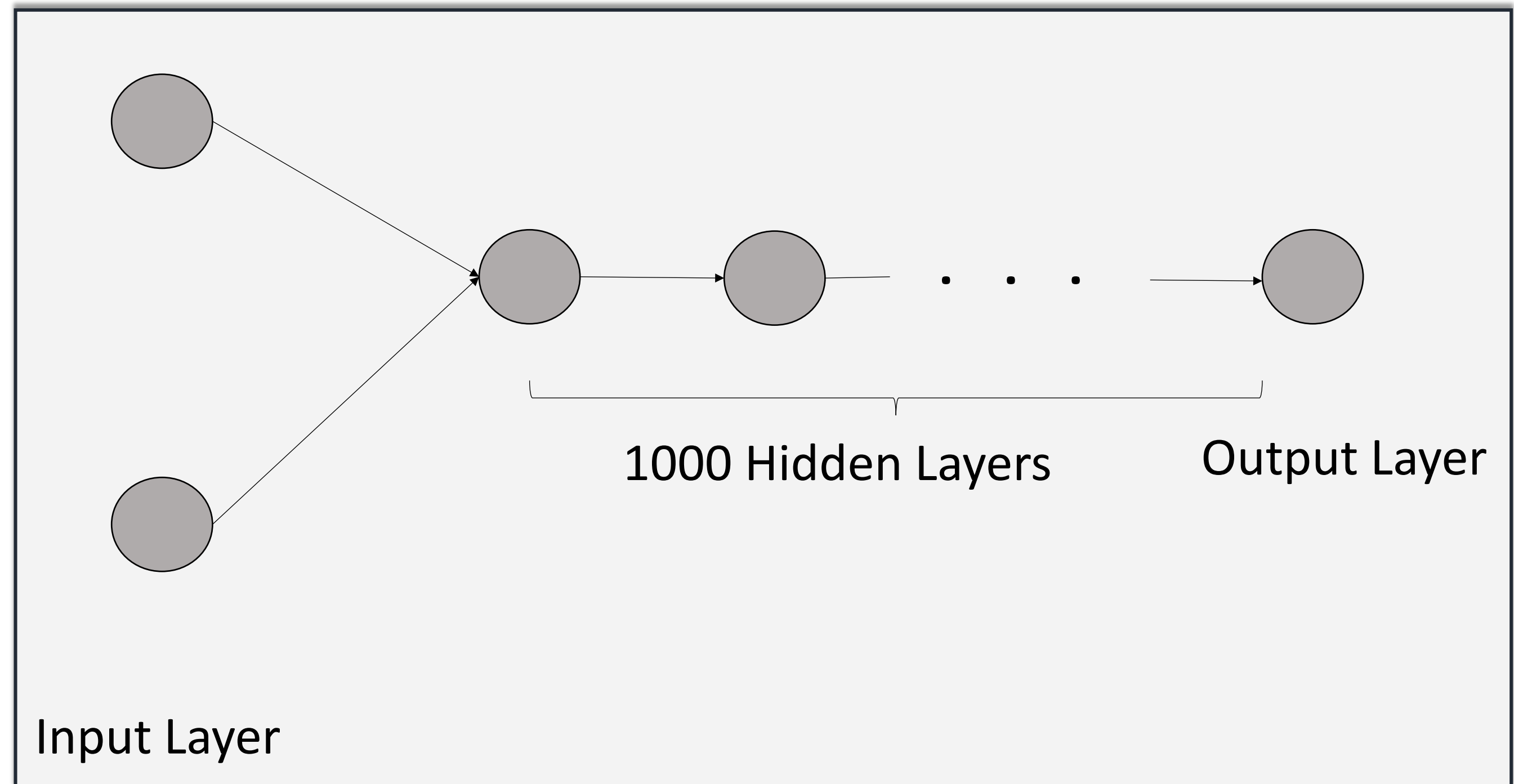
Depth= 2

Deep Learning Architectures

Question: If I put a thousand neurons in a single layer, or have a thousand layers each with one neuron, which one qualifies as deep learning?



Input Layer Output Layer, # neurons= 1000



Deep Learning Architectures

Case 1: One layer with many neurons:

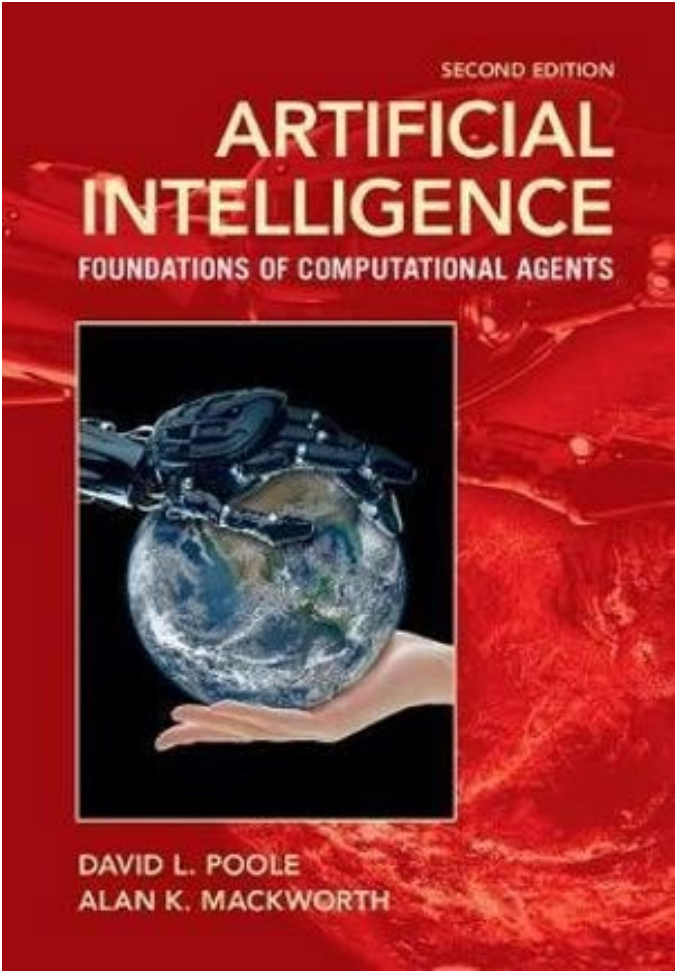
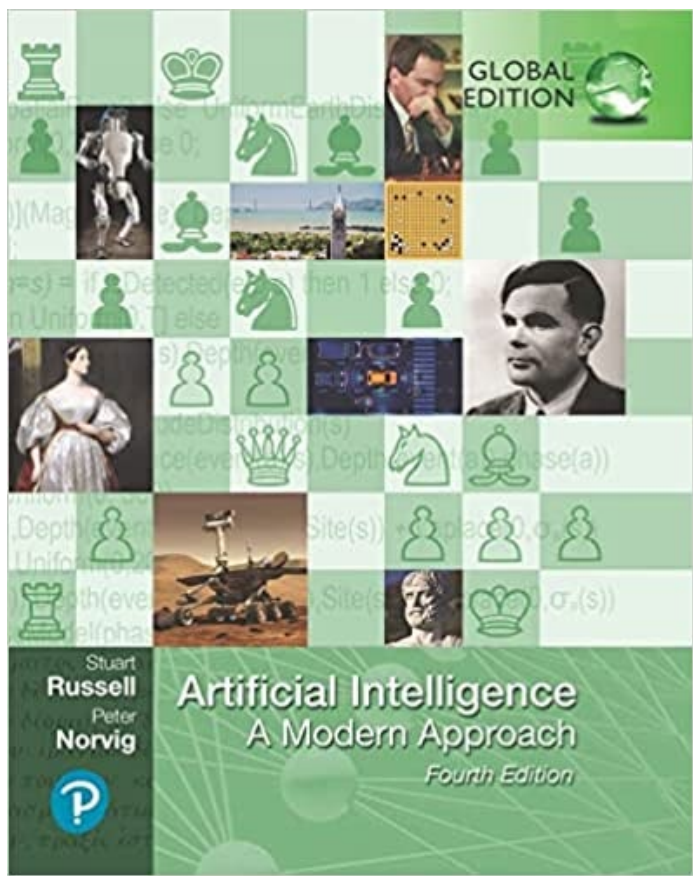
- This network has a very **wide hidden layer**, but it is still **shallow** because the depth is low (only one hidden layer).
- According to the **Universal Approximation Theorem**, a single wide hidden layer can approximate any continuous function with an arbitrary precision degree.
- However, in practice, this approach is computationally inefficient and does not capture hierarchical representations effectively (low precision).

Case 2: Many layers, each with one neuron:

- This network is extremely **deep but very narrow**.
- It qualifies as **deep learning** because the depth (number of layers) is enormous.
- However, with only one neuron per layer, the network's expressive power is limited.

References

- Poole & Mackworth, Artificial Intelligence: Foundations of Computational Agents, Chapter 7.
- Russell & Norvig, Artificial Intelligence: A Modern Approach, Chapters 18.6, 18.7.
- Bishop, Neural Networks and Their Applications, Review of Scientific Instruments, 65(6): 1803-1832.



REVIEW ARTICLE

Neural networks and their applications

Chris M. Bishop
Neural Computing Research Group, Department of Computer Science and Applied Mathematics,
Aston University, Birmingham, B4 7ET, United Kingdom
(Received 16 August 1993; accepted for publication 1 March 1994)

Neural networks provide a range of powerful new techniques for solving problems in pattern recognition, data analysis, and control. They have several notable features including high processing speeds and the ability to learn the solution to a problem from a set of examples. The majority of practical applications of neural networks currently make use of two basic network models. We describe these models in detail and explain the various techniques used to train them. Next we discuss a number of key issues which must be addressed when applying neural networks to practical problems, and highlight several potential pitfalls. Finally, we survey the various classes of problem which may be addressed using neural networks, and we illustrate them with a variety of successful applications drawn from a range of fields. It is intended that this review should be accessible to readers with no previous knowledge of neural networks, and yet also provide new insights for those already making practical use of these techniques.

TABLE OF CONTENTS	
I. INTRODUCTION.....	1803
A. Overview of Neural Networks.....	1804
B. Biological Neural Networks.....	1804
C. Artificial Neural Networks.....	1805
D. A Brief History of Neural Computing.....	1806
II. MULTIVARIATE NON-LINEAR MAPPING.....	1806
A. Analogy with Polynomial Curve Fitting.....	1807
B. Error Functions and Network Training.....	1807
C. Interpolation and Classification.....	1808
III. THE MULTILAYER PERCEPTRON.....	1808
A. Architecture of the Multilayer Perceptron.....	1808
B. Network Training.....	1811
C. Gradient Descent.....	1813
D. Alternative Training Algorithms.....	1814
IV. RADIAL BASIS FUNCTION NETWORKS.....	1815
A. Structure of the Radial Basis Function Network.....	1815
B. Choosing the Basis Function Parameters.....	1816
C. Choosing the Second-layer Weights.....	1817
V. LEARNING AND GENERALIZATION.....	1817
A. Interpretation of Network Outputs.....	1818
B. Generalization.....	1819
C. Determination of Network Topology.....	1820
VI. DATA PREPROCESSING.....	1821
A. The Curse of Dimensionality.....	1821
B. Linear Rescaling.....	1822
C. Feature Extraction.....	1822
D. Prior Knowledge.....	1823
VII. IMPLEMENTATION OF NEURAL NETWORKS.....	1823
A. Software Implementation.....	1823
B. Hardware Implementation.....	1823
VIII. EXAMPLE APPLICATIONS.....	1824
A. Interpolation.....	1824
B. Classification.....	1826
C. Inverse Problems.....	1827
D. Control Applications.....	1829
IX. DISCUSSION.....	1830
A. Other Network Models.....	1830
B. Future Developments.....	1830
APPENDIX: A GUIDE TO THE NEURAL COMPUTING LITERATURE.....	1830

I. INTRODUCTION

Since the late 1980's there has been a dramatic growth in the level of research activity in neural networks, accompanied by extensive coverage in the popular press. While much of the research effort has been concerned with developing fundamental principles and new algorithms, there has also been an increasing drive towards real-world application. Indeed, the last few years have seen the subject mature to the point where numerous practical applications are in routine use across a range of fields. It is now clear that neural networks offer a powerful set of tools for solving problems in pattern recognition, data processing, and non-linear control, which can be regarded as complementary to those of more conventional approaches. Scientific instrumentation in particular is one area where there is an increasing need for fast non-linear methods for data processing, and where neural network techniques have much to offer.

Of the many neural network models which have been developed, we shall focus primarily on two, known respectively as the multilayer perceptron and the radial basis function network. These networks presently form the basis for the

Feedback

- In case you want to provide anonymous feedback on these lectures, please visit:
- <https://forms.gle/KBkN744QuffuAZLF8>

Muchas gracias!



AI Lecture Feedback

This is a short form to provide early feedback for lectures

franciscocruzhh@gmail.com [Switch account](#)

Not shared

* Indicates required question

In case you want a reply, provide your zID. Otherwise your answer is anonymous.

Your answer

how did you participate? *

☐ In the classroom

☐ Watch the class from automatic recording

If you have any comments, feedback, or question about the lectures, this is the place. *

Your answer

Submit Clear form